VIT VELLORE · SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BCSE307L – Compiler Design

FINAL PROJECT REPORT · REVIEW 3

PYTHIA and PYTHIA Studio

A verified source-to-source accelerator that converts the slow parts of a Python project to C++17, checks every conversion against CPython, and splices it back in

Item	Details
Course	BCSE307L – Compiler Design
Project ID and title	A15 – Typed Source-to-Source Transpiler
Team number	Team 2
Review	Review 3 (final), October 2026
Faculty	Shalini L
Repository state	branch `main`, commit `95bbb1b` (143 commits)

Team member	Register number	Role
Vishal Vivek (team lead)	24BCE2377	Front end, integration, repository, Studio application
Rujuta Mangesh Kulkarni	24BDS0459	Type checker, semantic analysis, dictionary support, negative corpus
Koliparthy Venkata Jahnavi	24BDS0132	Lowering, divergence catalogue, runtime, guard elision, PySet
Aaryan Gupta	24BDS0134	C++ emitter, C11 to C++17 migration, differential oracle, CI, corpus

Declaration. We declare that this report and the project it describes are the work of the team named above. Where we reused open-source software, documentation or AI coding assistants, we have said so in Section 14. Every number in this report comes from a command we ran on the machine described in Section 5; Appendix F lists each claim with its command, result and date.

Signatures: _____ _____ _____ _____

Contents

1. Abstract 3
2. Introduction and motivation 3
3. Problem statement, objectives, scope and limitations 3
4. Background and related work 5
5. Requirements and technology stack 6
6. System design 7
7. Algorithms and methodology 10
8. Implementation details 13
9. Testing strategy and test cases 13
10. Results and evaluation 15
11. Error handling, limitations and known issues 17
12. Member-wise contribution 19
13. Conclusion and future work 19
14. References 20
Appendix A. User manual 22
Appendix B. Installation guide 23
Appendix C. Sample runs 24
Appendix D. Complete test table 28
Appendix E. Grammar specification and API specification 32
Appendix F. Evidence register 34
Appendix G. Contribution log 36
Appendix H. User-interface screenshots and additional diagrams 37

1. Abstract

Python is easy to write and slow on loop-heavy code, and the usual remedies either cost a second code base or give no evidence that the fast version behaves like the original. PYTHIA is a source-to-source compiler and desktop application that converts the hot functions of an ordinary Python project to C++17, checks every conversion against CPython on the program's own recorded calls, and splices the survivors back in behind a one-line decorator. The compiler core (off-side-rule lexer, Pratt parser, bidirectional type checker, lowering with a fourteen-class Semantic Fidelity Report, interval-based guard elision, C++17 emitter) is extended by an in-house type inference engine for untyped code that uses no language model, a tracer and hotspot ranker, pybind11 library mode, and a replay-based verifier. PYTHIA Studio shows every stage and installs as a 90.0 MiB Windows package. On the machine described in Section 5 the differential oracle agrees with CPython on 78 of 78 programs and rejects 88 of 88 invalid ones; 91 of 91 untyped everyday programs convert and match without any LLM. On ten fixed kernels the compiled binaries are 1.5 to 30 times faster than CPython on nine (median over all ten 11x) with identical output, and about 29 times slower on one dictionary-heavy kernel. Through the full Studio pipeline on a small public repository (dayyass/project-euler), a hot function ran 21.3x faster, verified on 500 of 500 recorded calls, and the whole program went from 8.9 s to 0.8 s with identical output. The limits are stated: 64-bit integers, ASCII strings, no C-extension libraries, and verification that is differential testing rather than proof. The WebAssembly back end promised at Review 2 was dropped in favour of the Studio pipeline.

2. Introduction and motivation

Python is easy to write and slow on loop-heavy code, because CPython handles every value as a heap object. The usual remedies each move the problem: a rewrite in C++ costs a second code base, a dialect such as Cython costs a new syntax, a JIT constrains the code, and an LLM translation comes with no evidence that it behaves like the original.

What a working developer actually has is an untyped project and a profile saying three functions take most of the time. They want those functions faster, without a rewrite and without trusting a black box. That is a compiler problem: read the code (front end), know what it means (types), make Python's implicit behaviour explicit (IR), produce native code (back end), and argue why the output may replace the input (verification).

- **Beneficiaries:** developers with a slow Python project; teams auditing a port (the Fidelity Report is the audit trail); students of compiler design.
- **Relevance to the course:** lexing with the off-side rule, Pratt parsing, scoped symbol tables and bidirectional type checking, type inference over a lattice, lowering, interval analysis, syntax-directed code generation and translation validation, plus the practical parts textbooks skip (host-runtime interfacing, build caching, packaging).

3. Problem statement, objectives, scope and limitations

Problem. *Given a Python project, find where its time goes, translate each function that can be translated into C++17, prove on the program's own recorded calls that the translation behaves like CPython, install only the translations that pass and are actually faster, and report every place where the two languages could differ.* Functions that cannot be translated keep running in Python, with a plain-English reason.

Scope. In: the PySub language (a typed Python subset grown to classes, dicts, sets, tuples, comprehensions, exceptions, f-strings and part of the standard library), an in-house converter from untyped Python, the C++17 back end and runtime, pybind11 library mode, the Studio application and a Windows installer. Out, by design: C-extension libraries (NumPy, pandas, PyTorch), generators, `eval`/`exec`, threads, file and network I/O, regex, unseeded randomness, non-ASCII text, and any back end other than C++17.

3.1 What we achieved

- **A complete compiler pipeline** for PySub: off-side-rule lexer, Pratt parser, bidirectional type checker, lowering to PIR, interval-based guard elision and a C++17 emitter with its runtime.
- **Correctness against CPython:** 78 of 78 valid programs give identical output and exception class, and 88 of 88 invalid programs are rejected with the intended message.
- **A Semantic Fidelity Report** with 14 divergence classes; 1280 positioned notes over the test corpus.

- **In-house type inference, no LLM:** 91 of 91 untyped everyday programs convert, compile and match CPython.
- **Speed:** on ten fixed kernels the compiled code is a median 11x faster than CPython with identical output (nine of the ten are 1.5x to 30x).
- **PYTHIA Studio:** a ten-stage pipeline that scans, traces, ranks, types, converts, compiles, replay-verifies, benchmarks and splices a real project, with a Windows installer.
- **A real project, end to end:** on a small public repository the whole program ran 11.6x faster with identical output (Section 10.4).

3.2 Review 2 to Review 3: what changed

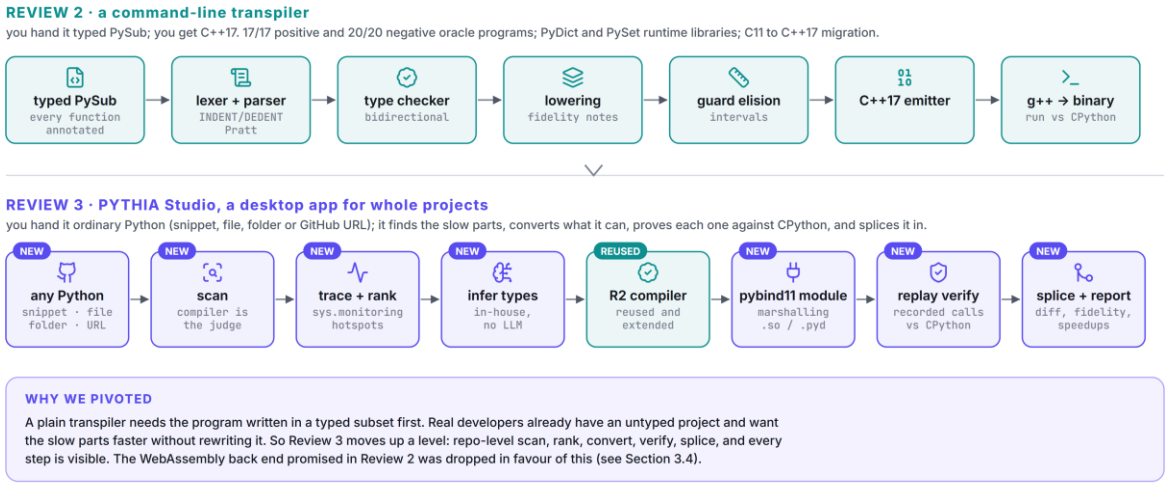


Figure 3.1. Review 2 (a command-line transpiler) and Review 3 (the Studio application).

Table 3.1. What changed since Review 2.

	Review 2	Review 3
Input	Hand-annotated PySub	Ordinary Python: snippet, file, folder or GitHub URL; types inferred
Unit	One whole program	One function at a time inside a project, spliced back behind a decorator
Types	Written by the author	In-house inference (observe, fixpoint, search); no LLM needed
Evidence	Oracle: 17 positive, 20 negative programs	Oracle 78 and 88, plus replay of the program's own calls for every converted function
Language	Scalars, lists, PyDict, PySet	Plus classes, tuples, comprehensions, try/except, f-strings, a standard-library subset
Interface	Command line	Studio: pipeline stepper, diff with fidelity gutter, function journey, playground, report
Delivery	Clone and run	Windows installer, portable zip, wheel, Docker image
Back ends	C++17 (WebAssembly promised)	C++17 only; **WebAssembly dropped**

Why we changed direction. A transpiler that takes hand-annotated PySub is a good compiler exercise, but nobody has a PySub program; they have a project. They need to find the hot functions, type untyped code, and trust the result. So the compiler core stayed and the new work sits around it: scan, trace and rank, in-house inference, replay verification, splice, and an application that shows every step.

Targets that changed.

- **WebAssembly back end: dropped.** It would show retargetability but not help a developer speed up a Python project; the time went to Studio.
- **Oracle corpus:** target 120 positive and 60 negative. Negative met (88); positive is 78, backed by an independent 91-program untyped gate and a generator (Section 9).
- **Reclamation:** delivered for library mode (a per-call arena); standalone binaries still never free memory (D7).

4. Background and related work

Concepts. The off-side rule turns indentation into INDENT/DEDENT tokens. Pratt parsing handles expressions with a binding-power table. Bidirectional type checking pushes an expected type inward and synthesises types bottom-up. Interval abstract interpretation (Cousot and Cousot) gives sound integer ranges. Differential testing (McKeeman) compares two implementations on the same input; applied to a translator it is a form of translation validation: the output is checked per program, not proved correct for all programs.

4.1 Prior tools

Figure 4.1 compares nine existing systems with PYTHIA on the criteria that matter here; each cell follows the tool's own documentation (Section 14). Several of them beat PYTHIA in ways that matter:

TOOL · OUTPUT	INPUT IT NEEDS	Readable output	Checked vs CPython	Reports divergence	Whole repo automatic	Unconvert-ible code still runs	No LLM or internet	Visual stage app
Cython C/C++ extension module	Python + C type declarations	○	○	○	○	●	●	○
Numba JIT machine code (in memory)	Python + @jit, NumPy style	–	○	○	○	○	●	○
PyPy alternative interpreter + JIT	unmodified Python	–	○	○	●	◐	●	○
mypyc C extension module	type-annotated Python	○	○	○	◐	●	●	○
Nuitka C program linked to libpython	any Python source	○	○	○	●	●	●	○
Codon native executable (LLVM)	Python-like, not drop-in	○	○	○	○	◐	●	○
Pythran C++ native module	Python subset + export comments	–	○	○	○	○	●	○
Shed Skin C++ program or module	implicitly typed Python 3 subset	–	○	○	○	○	●	○
LPython LLVM / C / C++ / WASM (alpha)	type-annotated Python subset	–	○	○	○	○	●	○
LLM translation C++ source text	any Python	●	○	○	○	○	○	○
PYTHIA readable C++17 + pybind11 module	untyped Python, supported subset	●	●	●	●	●	●	●

● yes ◐ partly / with caveats ○ no – not applicable or not assessed

Figure 4.1. Comparison with existing tools. Filled circle: yes; half: partly or with caveats; empty: no; dash: not applicable. "Readable output" is the team's own assessment.

- **PyPy** runs unmodified programs with high compatibility at about 4x CPython on average (its own site); **Numba** needs only a decorator and excels on array code.
- **Cython, mypyc, Nuitka** handle far more of Python and its libraries; none checks its output or reports divergence.
- **Codon, Pythran, Shed Skin, LPython** compile a subset to native code and can beat our output, but need code written for them.
- **LLM translation** accepts any code and gives readable output, but nothing checks it, results vary between runs, and it needs a network.

4.2 The gap and our novelty

Tools that accept untyped code do not give a readable, auditable artefact; tools that give readable native code need code written for them; none checks its output against CPython or reports where semantics may differ. PYTHIA's contribution is the combination, stated only as far as Section 10 supports:

1. **Verified, not asserted.** Each converted function is replayed against CPython on calls recorded from the user's own program (topped up with labelled generated inputs); a function that disagrees is rejected automatically. It is differential testing, not a proof.
2. **Semantic Fidelity Report.** A positioned list of every site where Python and C++ could differ, with class (D1 to D14) and mitigation; we found no surveyed tool documenting one.
3. **Whole project, hands-off.** Scan, rank, convert what can be converted, explain the rest in plain English, splice back while the project keeps running.
4. **In-house type inference, no LLM.** Deterministic, free, offline. An optional LLM fallback exists, is off without a key, and cannot bypass the compiler or the replay.
5. **Readable, exportable C++17** and an application that shows every stage.

We do not claim to be faster than Codon or Pythran on code they accept, to cover as much of Python as PyPy, Cython or Nuitka, or that "verified" means more than "agreed with CPython on the calls we replayed".

5. Requirements and technology stack

Table 5.1. Environment used for every measurement in this report.

Item	Value (detected on the machine that produced the results)
Machine	Intel(R) Core(TM) Ultra 9 185H (16 cores, 22 threads), 15.5 GiB RAM
Operating system	Microsoft Windows 11 Home Single Language 10.0.26300; WSL2 Ubuntu, kernel 6.6.87.1-microsoft-standard-WSL2
Python	Windows: 3.13.16 (the interpreter bundled in the installer); WSL: 3.14.4 (oracle, benchmarks)
C++ compiler	Windows: g++.exe (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r2) 16.2.0; WSL: c++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Flags	`-std=c++17 -O2 -flto` (oracle and kernels); `-O3 -flto -fPIC -fvisibility=hidden` (extension modules)
Libraries	pybind11 3.1.0; FastAPI 0.115.12, Uvicorn 0.34.3, Pydantic 2.11.5; Node 22.14.0, React 18.3.1, Vite 5.4.21, TypeScript 5.9.3, Monaco 0.52.2
Capture tools	Playwright 1.61.0 driving Microsoft Edge; Matplotlib 3.10.8; python-docx 1.2.0; Microsoft Word for the PDF export
Repository	commit `95bbb1b` on `main`; measurements on 2026-10-24

Table 5.2. Technology stack.

Layer	Choice (version used)	Role
Compiler	Python 3.13.16, standard library only	Lexer, parser, type checker, lowering, optimiser, emitter, converter
Target and runtime	C++17; `pyrt.hpp/.cpp`, `pydict.hpp`, `pyset.hpp`	Python-semantics runtime: int64 checks, floor division, strings, lists, dicts, sets, exceptions
Bridge	pybind11 3.1.0	Extension modules and marshalling (`pyrt_bridge.hpp`)
Toolchain	g++ (g++.exe (MinGW-W64 x86_64-ucrt-posix-seh on Windows)	Builds the generated source; bundled on Windows
Server	FastAPI 0.115.12, Uvicorn 0.34.3, WebSockets	REST, event stream, static UI
UI	React 18.3.1, TypeScript 5.9.3, Vite 5.4.21, Tailwind, Monaco 0.52.2, Zustand	Studio application; icons from Lucide
Packaging	Inno Setup 6.7.3, hatchling wheel, Docker	Installer, portable zip, wheel, container image
Test and measurement	oracle, `check_basics.py`, `check_snippets.py`, `property_test.py`, `fuzz.py`, `stress.py`	Section 9

Test inputs. All self-authored: 78 positive and 88 negative PySub programs, 91 untyped everyday programs, 22 LeetCode-style snippets, two bundled demo projects, a grammar-directed generator, a property-test harness and a stress harness (Section 9).

5.1 Why this, not that

Table 5.3. Technology choices. Where the reason is our own judgement, we say so.

Decision	Chosen	Considered	Why
Compiler language	Python 3	C++, Rust	The compiler manipulates trees and strings, not hot loops, and Studio must import and run the user's modules in process (tracing, replay). Our judgement.

Decision	Chosen	Considered	Why
Lexer and parser	Hand-written (`lexer.py`, `parser.py`)	ANTLR, Lark, PLY	Full control of the off-side rule and of positioned error recovery, and no third-party import in the compiler core. Cost: each new construct is a hand-written hook.
Target	C++17 (migrated from C11 on 4 Sept 2026, pull request 1)	C11, LLVM IR	pybind11 needs C++; templates give monomorphic containers; lambdas give single evaluation; the output stays readable source and g++ does the optimisation. C11 had needed GNU statement expressions at three sites (Review 1, Section 5.4). LLVM IR would not be readable and would need LLVM at run time.
Python bridge	pybind11 3.1.0	ctypes, cffi, raw C-API, nanobind	Automatic conversions and exception translation with one header-only dependency built by the same g++. ctypes and cffi need a C ABI and hand-written list and string marshalling; the raw C-API is long. We did not evaluate nanobind.
Correctness evidence	Differential testing (oracle and replay)	Unit tests, formal proof	The specification of a translator is "behaves like CPython", and the cheapest executable form of that is to run both. A proof of the whole pipeline was out of reach; we say so (Section 11).
Type source	In-house inference	LLM translation	We tried it first: a Groq/Anthropic fixer and an LLM Playground converter (commits `dfe6942`, `cf2cfb8`). The free tier is about 200,000 tokens per day per model (`STATUS.md`), answers vary between runs, it needs a network, and every answer still has to pass the compiler and the replay. The LLM path remains as an optional fallback (commit `e4a4804`).
Server	FastAPI with WebSockets	Flask, Django	First-class async WebSockets for the event stream, Pydantic models, and one small ASGI app that serves the UI bundle too.
Desktop UI	React, Vite, TypeScript, Monaco in an Edge/Chrome `--app` window	Tkinter, Qt, Electron	Monaco gives a real diff editor; the same UI runs in any browser and as a PWA; no second runtime to ship, so the installer is 90.0 MiB with Python and a compiler inside. We did not benchmark Electron.
Windows toolchain	Bundled MinGW-w64 GCC 16.2 (WinLibs, UCRT)	MSVC	The build flags (`-O3 -flto -fvisibility=hidden`) and the runtime are developed against GCC, and a redistributable zip can be shipped inside the installer; MSVC Build Tools are a separate installation. Our judgement.
Installer	Inno Setup 6.7.3	WiX / MSI, NSIS	One scripted `.iss`, per-user install without administrator rights, free.
Tracing	`sys.monitoring` (3.12+), `sys.setprofile` fallback	`sys.settrace`	Per-function call events at lower cost than line tracing; `settrace` is not used anywhere in the repository (`_tracer_boot.py:66`, `observe.py`).
Accept rule	Measured replay speedup of at least 1.1x	Static heuristics only	A function that compiles is not necessarily faster; the number decides (`DECISIONS.md`, B2).

6. System design

PYTHIA has two tiers. The **compiler** (`pythia/`) turns PySub into C++17. **Studio** (`pythia/studio/`) wraps it in a ten-stage pipeline that works on whole projects, with a web interface that shows each stage. Figure 6.1 draws the system in six layers; the numbered arrows are explained below it.

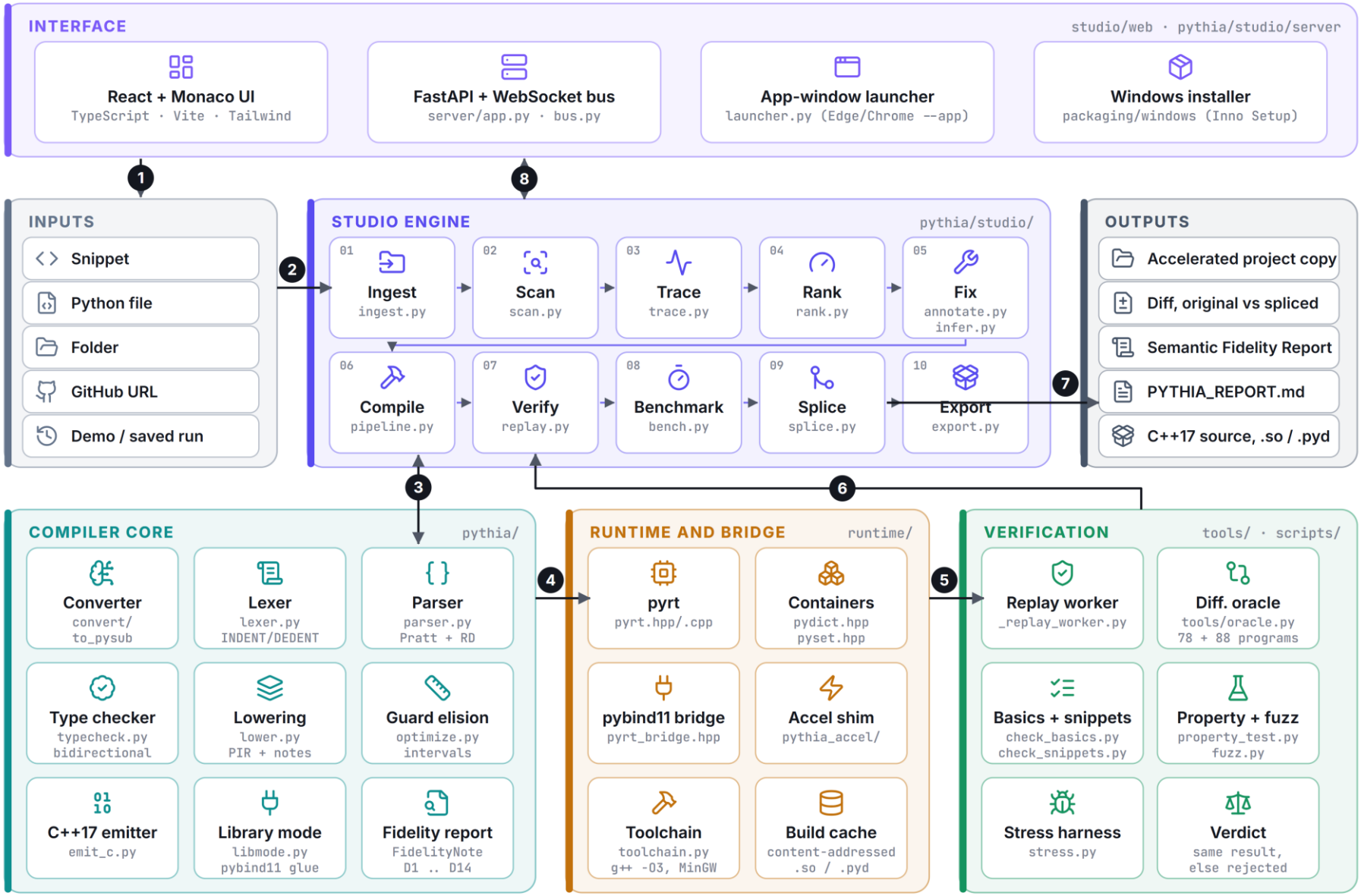


Figure 6.1. PYTHIA architecture. Arrows 1 to 8 are described in the text; stage numbers 01 to 10 are the pipeline order.

1. **Input.** A snippet, file, folder, GitHub URL, demo or saved run reaches the FastAPI server as a job.
2. **Ingest.** The project is copied to an isolated workspace (`original/` and `work/`); GitHub URLs are fetched as a zip, so git is not needed.
3. **Compiler as judge and builder.** Scan, Fix and Compile call the compiler core: the real parser and type checker decide eligibility, the converter adds missing annotations, and the lowering, guard elision, emitter and `libmode.py` produce C++17 and pybind11 glue.
4. **Build.** g++ builds a `.so` or `.pyd` against the runtime; builds are cached by content.
5. **Replay.** A worker process feeds the recorded calls to the original and the compiled function.
6. **Verdict.** Agreement and a speedup of at least 1.1x keep the function; anything else is rejected or declined with the numbers.
7. **Outputs.** Accepted functions get a one-line decorator in the working copy; the diff, fidelity report, `PYTHIA_REPORT.md`, C++ and modules are exported. The original folder is never modified.
8. **Interface.** Every stage emits sequenced events on a WebSocket bus, so the UI can play a run live, replay a saved one, or step through it.

6.1 Modules

Table 6.1. Main modules; line counts are non-blank lines (`collect_facts.py`).

Module	Files (lines of code)	Responsibility
Front end	`lexer.py`, `parser.py`, `parse_ext.py`, `parse_iter.py`, `parse_types.py`, `ast_nodes.py` (2,325)	Tokens with layout, AST, positioned syntax diagnostics with recovery
Type system	`typecheck.py`, `types_ext.py`, `classes_ext.py`, `builtins_ext.py`, `tuples.py`, `comprehensions.py` (6,136)	Scopes, bidirectional checking, classes, tuples, comprehensions, built-ins
Lowering and optimiser	`lower.py`, `optimize.py` (787)	PIR with explicit coercions, fidelity notes, interval guard elision
Back end	`emit_c.py`, `emit_ext.py`, `lib_ext.py`, `libmode.py`, `classes_lib.py`, `toolchain.py`, `cli.py`, `driver.py`, `stages.py` (3,006)	C++17 emission, pybind11 glue, builds, command line
Converter	`pythia/convert/` (2,687)	Untyped Python to PySub: observation, fixpoint inference, search
Runtime	`runtime/` (2,853)	Python semantics in C++, containers, marshalling, accel shim
Studio back end	`pythia/studio/` including server (7,307)	Ingest, scan, trace, rank, fix, compile, verify, benchmark, splice, export; REST and WebSocket
Studio UI	`studio/web/src` (9,531, TypeScript)	Stepper, editors, diff, journey, playground, report
Tools and scripts	`tools/`, `scripts/` (3,754)	Oracle, generator, checks, stress, property tests
Packaging	`packaging/windows/` (448)	Installer assembly, launcher, Inno Setup script

6.2 Key data structures and decisions

- **Token, AST, `Ty`:** tokens carry line and column; every AST node has a `ty` slot the type checker fills, so the typed tree is the same tree. Types widen bool, int, float (`widens_to`, `typecheck.py:54`).
- **FidelityNote** (`lower.py:26`): position, divergence class, detail, mitigation, severity (exact, guarded or divergent).
- **FunctionRecord** (`studio/models.py`): one per function, with eligibility, status (`verified`, `guarded`, `fixable`, `not_eligible`, `declined`, `failed`), trace, every source form, build, replay, benchmark, splice and the 16-step journey the UI shows.
- **The compiler is the judge.** `scan.py` does not list PySub's features; it extracts each function and asks the real parser and type checker.
- **Splice by decorator, never by rewrite.** The original body stays; if the compiled module is missing or raises `FallbackError`, Python runs.
- **Never splice a slowdown.** Below 1.1x measured, a function is `declined`; functions under 0.2% of run time are not even built.
- **CPython-exact or rejected.** Accepted code gives identical stdout and exception type and message, or the compiler refuses it with a reason.

7. Algorithms and methodology

For each stage: what it does, the algorithm, its cost and the trade-off we accepted. Figure 7.1 follows one real function through the stages; the numbers are from Section 10.4.

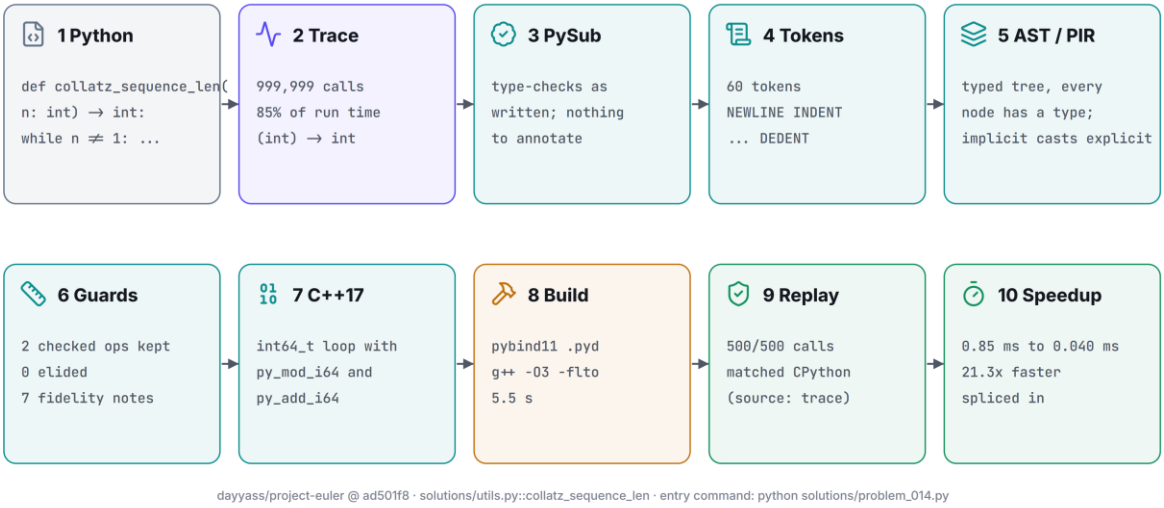


Figure 7.1. One function's journey: dayyass/project-euler, collatz_sequence_len, from Python to a replay-verified speedup.

7.1 Lexing and the off-side rule (`lexer.py`)

A stack of indentation widths starts as `[0]`. At the start of a logical line a larger indent pushes and emits INDENT; a smaller one pops and emits one DEDENT per level, and must land on a width still on the stack. Blank and comment lines carry no layout, brackets suppress it, `;` separates statements, and operators match longest-first. **Cost:** O(n). **Trade-off:** tabs are expanded to four spaces by the converter, not interpreted by the lexer.

7.2 Parsing (`parser.py`)

Statements use recursive descent; expressions use a Pratt loop (`parse_expr`, line 512) over a binding-power table (`or` 1 up to `**` 16, right associative). Chained comparisons desugar to `a < b and b < c`. Syntax errors use panic-mode recovery: `synchronize()` skips to the next statement boundary and always consumes a token, so one run reports several errors and cannot loop. **Cost:** O(n). **Trade-off:** hand-written, so each new construct is a hand-added hook; the gain is exact diagnostics and no dependency.

7.3 Bidirectional type checking (`typecheck.py`)

`synth` computes a type bottom-up; `check_expr(e, expected)` pushes one inward. That is what types an empty `[]` from its annotation, and widens `[1, 2.5]` against `list[float]`. Signatures are collected before bodies, so mutual recursion works; scopes are a parent-linked chain. **Cost:** O(n) visits with O(depth) lookups. **Trade-off:** one fixed type per variable, which gives unboxed fast code and rejects code that rebinds a name to another type.

7.4 In-house type inference for untyped Python (`pythia/convert/`)

1. **Observe** (`observe.py`): run the program once under `sys.setprofile` and record the runtime types of arguments and locals.
2. **Fixpoint** (`infer.py`, `Engine`): every variable, return, attribute and record field is a slot; evidence from assignments, calls, returns and iteration is joined on a type lattice until nothing changes. Conflicts become an absorbing `bad`; int-or-float becomes `num`, which is refused where it would change printed output.
3. **Search** (`search.py`): for parameters nobody calls, try up to 400 type assignments ordered by syntactic hints; the first that makes the whole program type-check wins.
4. **Judge:** the real PySub checker accepts or rejects the result; rejections come back with a plain-English reason.

Cost: linear passes, finite lattice height (nesting cut at depth 5), so it terminates. **Trade-off:** soundness over coverage; `coin_change` is refused because converting it would print `3.0 -1.0` where Python prints `3 -1` (Section 9.2). Figure 7.2 shows the ladder applied to each function.

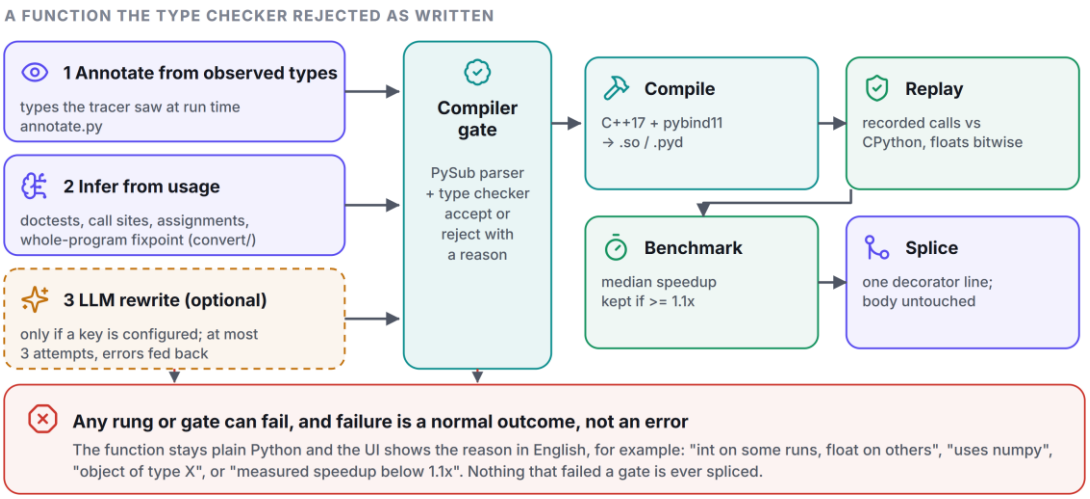


Figure 7.2. The fix-and-verify ladder. Every rung is deterministic except the optional last one, and none can bypass the compiler or the replay.

7.5 Lowering and the divergence catalogue (`lower.py`)

`Lowerer` rewrites the typed tree into PIR: every implicit conversion becomes an explicit `Coerce`, and every risky site gets a `FidelityNote`. Table 7.1 lists the fourteen classes. On the 78 positive programs the compiler emitted 1280 notes (545 exact, 726 guarded, 9 divergent); Section 10.3 breaks them down.

Table 7.1. The divergence catalogue (D1 to D14) as implemented.

Class	Divergence	Treatment
D1	Unbounded `int` against `int64_t`	Guarded: checked `+ - *` raise `OverflowError`; guards removed where interval analysis proves them unreachable; ints beyond int64 at the boundary raise `OverflowError`.
D2	`//` floors toward minus infinity, C++ truncates	Exact: `py_floordiv_i64` and the float form.
D3	`%` takes the sign of the divisor	Exact: `py_mod_i64`, including signed zero for floats.
D4	`/` is always true division	Exact: operands widened to double, `ZeroDivisionError` preserved.
D5	Negative indexing and bounds checks	Exact: index normalised and checked in the runtime; `IndexError` reproduced.
D6	`int ** negative int` changes the result type	Divergent: cannot be typed statically; reported at compile time, refused at run time.
D7	Object lifetime and reclamation	Guarded: standalone binaries never free; library mode frees everything allocated in a call when the call ends.
D8	Exception propagation across calls	Exact: C++ exceptions of one type (`PythonException`) caught in `main`. The note text in `lower.py` still describes the Review 1 `setjmp`/`longjmp` mechanism and is stale.
D9	Class fields zero-initialised before `__init__`	Guarded: an unset field reads as zero instead of raising `AttributeError`.
D10	`sum()` of an empty `list[float]`	Guarded: CPython returns int 0, compiled code 0.0; non-empty sums use the same compensated summation.
D11	Ordering of NaN in sort, min, max	Guarded: merge sort and timsort may place NaN differently.
D12	Interactive `input()`	Exact: prompt to stdout, one line from stdin, `EOFError` at end.
D13	Characters are ASCII	Guarded: non-ASCII input raises `FallbackError` at the boundary; `ord`/`chr` above 127 raise `PythiaSemanticError`.
D14	libm last-bit differences; user-defined ordering	Guarded: math functions call the toolchain's libm; sorting objects by `__lt__` calls it in a different order.

7.6 Guard elision by interval analysis (`optimize.py`)

Python ints do not overflow; int64 does. PYTHIA compiles `+ - *` as checked calls (`py_add_i64` and friends) that throw `OverflowError`, and removes a check only where an interval proof shows it cannot fire. Literals give point intervals, `range` and `len()` bound indices, results must stay within plus or minus 2^62, and loop-carried accumulators are widened to *unknown* before the body is analysed, which is why one forward pass is sound. It may

keep an unneeded check, never drop a needed one. **Cost:** O(n). **Measured effect:** none we can demonstrate (Section 10.2).

7.7 C++17 emission (`emit_c.py`)

One syntax-directed walk of PIR. Primitive lists map to monomorphic runtime types (`PyListI64`, `PyListF64`, ...), scalar dicts to specialised dict types, classes to `std::shared_ptr`. A `for ... in range` loop runs on a hidden counter so reassigning the loop variable cannot change the iteration; expressions that must evaluate an operand once use immediately invoked lambdas. Listing 7.1 is real output.

```
#include "pyrt.hpp"

/* forward declarations (PySub allows mutual recursion) */
static int64_t f_total(PyListI64 *p_xs, int64_t p_n);

static int64_t f_total(PyListI64 *p_xs, int64_t p_n) {
    int64_t p_s = 0;
    int64_t p_i = 0;


    p_s = INT64_C(0);
    {
        int64_t _t1 = p_n;
        int64_t _t2 = 1;
        if (_t2 == 0) py_raise("ValueError", "range() arg 3 must not be zero");
        for (int64_t _t3 = INT64_C(0);
             _t2 > 0 ? _t3 < _t1 : _t3 > _t1;
             _t3 += _t2) {
            p_i = _t3;
            p_s = py_add_i64(p_s, py_mul_i64(PyListI64_get(p_xs, p_i), INT64_C(2)));
        }
    }
    return p_s;
    return 0;   /* unreachable unless the result is Optional */
}

static int pythia_main(void) {
    pyrt_set_py_minor(13);   /* match this CPython's error texts */
    try {
        py_print_begin();
        py_print_sep();
        py_print_i64(f_total([&]() { PyListI64 *_t5 = PyListI64_new(); PyListI64_append(_t5, INT64_C(1));
PyListI64_append(_t5, INT64_C(2)); PyListI64_append(_t5, INT64_C(3)); return _t5; }(), INT64_C(3)));
        py_print_end();
        return 0;
    } catch (const PythonException &exc) {
        py_print_exception(exc);
        return 1;
```

Listing 7.1. `python -m pythia.cli e.py --emit-only` for a three-line kernel. The loop-carried `p_s` keeps its checks.

7.8 Library mode and marshalling (`libmode.py`, `pyrt_bridge.hpp`)

A module without `main()` gets a generated pybind11 wrapper. Arguments are copied in, results copied out, and mutated lists and dicts are written back. What cannot be represented faithfully (non-ASCII string, wrong-typed argument) raises `FallbackError` and the original Python runs; an int beyond int64 raises `OverflowError`. Each call runs in an arena that frees everything it allocated, and `print` inside compiled code is routed to Python's `sys.stdout` so output order matches. Builds are cached by content. **Trade-off:** copying makes large arguments expensive; the ranker accounts for it.

7.9 Tracing and ranking (`_tracer_boot.py`, `rank.py`)

The entry command runs with a tracer injected as `sitecustomize`: `sys.monitoring` on Python 3.12+, `sys.setprofile` before. It records calls, time, argument and return types and sampled call records. The ranker scores `trace_share x eligibility x expected_benefit x (1 - boundary_penalty)`, with `marshal = calls x (1.5 us + 4 ns x elements)`; a function with negative predicted net saving is declined, and those under 0.2% of run time are marked cold. The two constants are calibration knobs.

7.10 Oracle and replay verification (`tools/oracle.py`, `replay.py`)

The **oracle** checks the compiler: each positive program runs under CPython and as a binary and must give byte-identical stdout and exception class; each negative program must be rejected with the `# expect:` text. The **replay**

checks one conversion: at least 50 calls (recorded, topped up and labelled) go through original and compiled function in a worker process; return value, exception and mutated arguments must match, floats by bit pattern. A match is then benchmarked and kept only if at least 1.1x faster. **Limit:** agreement on the replayed calls is evidence, not proof.

8. Implementation details

```
pythia/                     the compiler (Python)
  lexer.py  parser.py  ast_nodes.py    off-side rule, Pratt/recursive descent, AST
  typecheck.py  types_ext.py           bidirectional type checker and extensions
  classes_ext.py  tuples.py  comprehensions.py  builtins_ext.py
  lower.py  optimize.py                PIR, fidelity notes, interval guard elision
  emit_c.py (emit_cpp.py alias)  emit_ext.py  lib_ext.py   C++17 emitter
  libmode.py  classes_lib.py           pybind11 extension modules
  toolchain.py  paths.py  cli.py  driver.py  stages.py
  convert/                           in-house untyped-Python converter (no LLM)
  studio/                            Studio back end: pipeline, scan, rank, replay, splice, server/
runtime/                    C++ runtime: pyrt.hpp/.cpp, pydict.hpp, pyset.hpp, pyrt_bridge.hpp, pythia_accel/
                            (pyrt.c/.h: Review 1 C11 runtime, not used by the C++17 pipeline)
studio/web/                 React + TypeScript UI (dist/ committed)    studio/DECISIONS.md  CONTRACTS.md
packaging/windows/          installer build (build.py, launcher.c, .iss)
tests/positive  tests/negative   oracle corpora (78 and 88 programs)
tools/                      oracle.py  fuzz.py  fuzz_run.py
scripts/                    check_basics, check_snippets, property_test, stress, check_libmode, ...
demo/                       quanta (3,800 lines), textlab (480 lines), offline cache, sample run
docs/                       grammar.ebnf, Review 1 report/deck, review3/ (this report, viva guide, sources)
.github/workflows/          ci.yml (oracle, library mode, package), release.yml
```

Listing 8.1. Repository layout (selected).

- **Front end and type system** grew by small extension modules registered on a few dispatch chains (`parse_ext.py`, `types_ext.py`, `classes_ext.py`, `tuples.py`, `comprehensions.py`, `builtins_ext.py`); those chains are where parallel work conflicted most.
- **Back end.** `emit_c.py` kept its Review 1 name through the C++17 migration; `emit_cpp.py` is an eight-line alias. The old C11 runtime (`pyrt.c/.h`) stays in the tree and is unused.
- **Studio.** `pipeline.py` runs the ten stages; `scan.py` extracts each function with its helpers; `replay.py`, `_replay_worker.py` and `codec.py` verify; `splice.py` and `export.py` produce the accelerated copy. The React UI is bundled with its fonts and committed as `dist/`.

Build and packaging. `python -m pythia.cli prog.py --run` parses, checks, lowers, emits and calls g++; `--fidelity` prints the report and `--lib f,g` builds a module. `packaging/windows/build.py` assembles a relocatable CPython 3.13, a pruned MinGW-w64 GCC 16.2 toolchain, the compiler, the UI and prebuilt demo modules, then Inno Setup 6.7.3 writes the installer (90.0 MiB, per-user, no administrator rights) and a 214.6 MiB portable zip. **CI** (`.github/workflows/ci.yml`) runs the oracle, a library-mode check and a packaging check on every push; we did not run CI in this session.

9. Testing strategy and test cases

Each layer catches a different kind of error. Suites are in Table 9.1; representative cases with expected and actual results are in Table 9.2.

Table 9.1. Test suites.

Suite	What it checks	Size	Result (this session)
Oracle, positive	Compiled binary against CPython: stdout and exception class	78 programs	78/78 identical
Oracle, negative	Compiler must reject, with the `# expect:` text	88 programs	88/88 rejected
Untyped basics (`check_basics.py`)	In-house conversion, compile, compare; no LLM	91 programs	91/91 identical
Snippets (`check_snippets.py`)	LeetCode and GfG style, untyped	22 snippets	21/22 (`coin_change` refused)
Library mode (`check_libmode.py`)	pybind11 module against CPython: values, exceptions, write-back, classes, leak	74 + 22 calls	OK
Grammar-directed generator (`fuzz.py`)	Sampled well-typed programs, oracle-style comparison	150	150/150 identical, 0 mismatches

Suite	What it checks	Size	Result (this session)
Property tests (`property_test.py`)	Ten algorithm categories, randomised inputs	10 categories	50/50 identical
Kernel benchmark	Ten fixed kernels, median of runs, outputs compared	10 kernels	outputs identical
Studio runs	Whole projects through the real pipeline with replay	5 runs	see Section 10.4
Stress (`stress.py`)	Public repositories file by file, no LLM	not run	cut: not re-run this session

9.1 Representative cases

Table 9.2. Representative cases; actual values are from the logged runs (Appendix F). LIMIT rows are documented limits, not failures.

ID	Category	Input / description	Expected	Actual	Status
TC-01	Positive	Operator precedence: print(2 + 3 * 4 ** 2)	CPython: 50	PYTHIA: 50	PASS
TC-02	Boundary	Floor division and modulo with negative operands	CPython: -4 1 -4 -1	PYTHIA: -4 1 -4 -1	PASS
TC-03	Boundary	Float repr: 0.1 + 0.2 and 1e16	CPython: 0.30000000000000004 1e+16 1000000000000000.0	PYTHIA: 0.30000000000000004 1e+16 1000000000000000.0	PASS
TC-04	Negative	Type mismatch in assignment: x: int = 'hello'	CPython: hello	PYTHIA: rejected at 1:10: 'x' is annotated int but the initialiser has type str	PASS
TC-05	Boundary	Index out of range: xs[10] on a 3-element list	CPython: [IndexError]	PYTHIA: [IndexError]	PASS
TC-06	Boundary	Division by zero: 1 // z	CPython: [ZeroDivisionError]	PYTHIA: [ZeroDivisionError]	PASS
TC-07	Boundary	range with step zero	CPython: [ValueError]	PYTHIA: [ValueError]	PASS
TC-08	Regression	Unbounded recursion	CPython: [RecursionError]	PYTHIA: [RecursionError]	PASS
TC-09	OOP	Class, method call, field update	CPython: 12	PYTHIA: 12	PASS
TC-10	Limit (D1)	Integer beyond int64: print(2 ** 62 * 2)	CPython: 9223372036854775808	PYTHIA: [OverflowError]	LIMIT
TC-11	Limit (D13)	Non-ASCII string length: len('héllo')	CPython: 5	PYTHIA: 6	LIMIT
TC-12	In-place	Library mode: swap list ends inside compiled code, caller's list	CPython: [4, 2, 3, 1]	PYTHIA: [4, 2, 3, 1]	PASS
TC-13	Limit (D13)	Library mode: word_len('héllo') with non-ASCII input	CPython: 5	PYTHIA: FallbackError (shim then runs the Python body)	PASS
TC-14	Limit (D1)	Library mode: list element 2**70 (beyond int64)	CPython: works (bignum)	PYTHIA: OverflowError	LIMIT

9.2 A regression we found and kept honest

`check_snippets.py` reported 21 of 22 snippets, not the 22 of 22 in `STATUS.md`. Bisecting with `git worktree` traced it to commit `210a599`. `coin_change` starts `dp` at `float('inf')` and returns `-1` when unreachable. Before the commit the converter typed it as returning `float`, which compiles but prints `3.0 -1.0` where CPython prints `3 -1`; now the converter reports conflicting types and refuses. The refusal is the CPython-exact behaviour, so we corrected the number, not the code.

Two bugs found while preparing the demo, fixed in commit `bd7e8ca`: `print` inside a compiled function bypassed Python's stdout (reordered and wrong newlines when piped, so a whole-program output hash differed), and replay failed on scripts that import siblings by bare name.

9.3 Stress

`scripts/stress.py` pushes every file of a public repository through converter, checker, build and run. It was **not re-run** for this report, so we make no stress claim.

10. Results and evaluation

10.1 Correctness

Table 10.1. Correctness results from this session (Appendix F has commands and dates).

Gate	Result	What it shows
Differential oracle, positive	78/78 (100%)	Compiled output and exception class equal CPython's on every program.
Differential oracle, negative	88/88 (100%)	Every invalid program is rejected with the intended message.
Untyped basics, zero LLM	91/91 (100%)	The in-house converter plus compiler reproduces CPython on everyday programs without annotations.
Snippets, zero LLM	21/22	One snippet is refused because converting it would change printed output.
Library mode	OK	74 calls and 22 class calls compared with CPython; no RSS growth over 3000 calls.
Generated programs	150/150 identical, 0 mismatches	Programs sampled from the grammar, compared with CPython.
Property tests	50/50 identical	Randomised inputs in ten algorithm categories.

Every gate reports agreement with CPython or an explicit refusal; none reports a silent difference. The one number that fell below its recorded value, 21 of 22 snippets, is a refusal: the refused program would have printed `3.0 -1.0` where CPython prints `3 -1`. The oracle's own speed column is one run of a whole process per program and mixes very short programs with long ones, so it is reported (median 15.2x) but the kernel benchmark below is used for performance claims.

10.2 Performance on ten fixed kernels

Ten kernels were fixed before measuring (`docs/review3/src/kernels/`): integer loops, sieve, matrix multiply, recursion, dynamic programming, floating-point iteration, sorting and a dictionary count. Each ran 5 times under CPython and as a native binary; the table gives the **median whole-process time** (CPython start-up is 104.6 ms of every Python figure). Outputs were byte-identical in every run.

Table 10.2. Kernel benchmark (medians; min and max are in `evidence/bench_kernels.json`).

Kernel	CPython (ms)	PYTHIA (ms)	Speedup	No guard elision (ms)	Elision ratio	Elided / kept
collatz	1822	66	27.4x	66	0.99x	0 / 2
sieve	381	42	9.1x	41	0.97x	2 / 1
matmul	237	20	11.6x	21	1.01x	1 / 9
nqueens	106	11	9.6x	9	0.85x	2 / 13
levenshtein	360	247	1.5x	238	0.96x	0 / 13
mandelbrot	507	17	29.6x	20	1.17x	0 / 0
fib	258	26	9.9x	28	1.09x	0 / 3
bubble	365	12	29.7x	14	1.15x	0 / 7
knapsack	295	20	14.5x	18	0.89x	1 / 7
wordcount	485	14260	0.03x	14242	1.00x	0 / 4

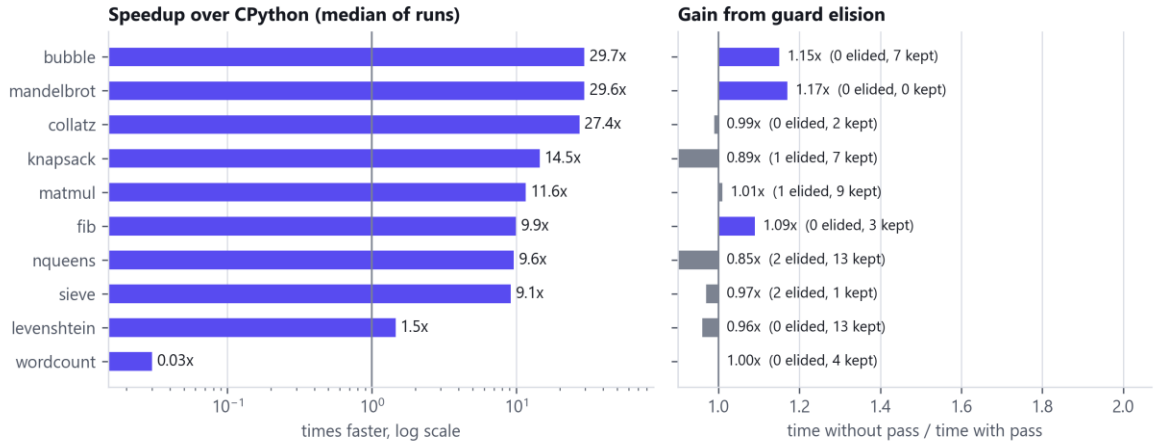


Figure 10.1. Speedup per kernel (left, log scale) and the effect of guard elision (right).

Guard elision. Nine of the ten kernels are 1.5x to 29.7x faster. Indexed integer and float loops (collatz, mandelbrot, bubble sort) gain most; recursion, the sieve, matrix multiply and N-queens gain about 9 to 12x; Levenshtein gains only 1.5x because the string-indexing path allocates. The geometric mean over all ten is 6.59x and the median 10.8x; every output was byte-identical to CPython's. **One kernel is a failure and we report it:** the string-keyed dictionary count (`k10_wordcount`, about 2,000 distinct keys, 1.5 million updates) ran in 14260 ms compiled against 485 ms in CPython, about 29 times slower. The cause is in the runtime: the eight specialised dictionary types look keys up by scanning a vector (`DICT_IMPL`, `runtime/pyrt.cpp:131`), so each lookup is O(number of keys), where CPython's is O(1). Dictionaries are correct but not fast; a hash index would fix it and is the first item in Section 13.2.

Guard elision. The right-hand panel of Figure 10.1 compares each kernel with and without the interval pass. The ratio lies between 0.85x and 1.17x, scattered on both sides of 1.0, which is the size of run-to-run noise at five runs. **We therefore cannot claim a measured benefit from the pass.** It removes very few checks on these kernels (Table Table 10.2), and with `-O2 -flto` the compiler can often inline the checked helpers and predict their branches. The Review 1 report anticipated a factor of two; we did not observe it.

10.3 Semantic Fidelity Report statistics

Over the 78 positive programs the compiler emitted 1280 positioned notes (median 11 per program): 42.6% exact, 56.7% guarded, 0.7% divergent. The divergent ones are all D6 (`int ** negative int` changes the type), which is reported at compile time and refused at run time. D1 (integer width) and D5 (indexing and bounds) dominate because every integer operation and subscript is a site (Figure 10.2).

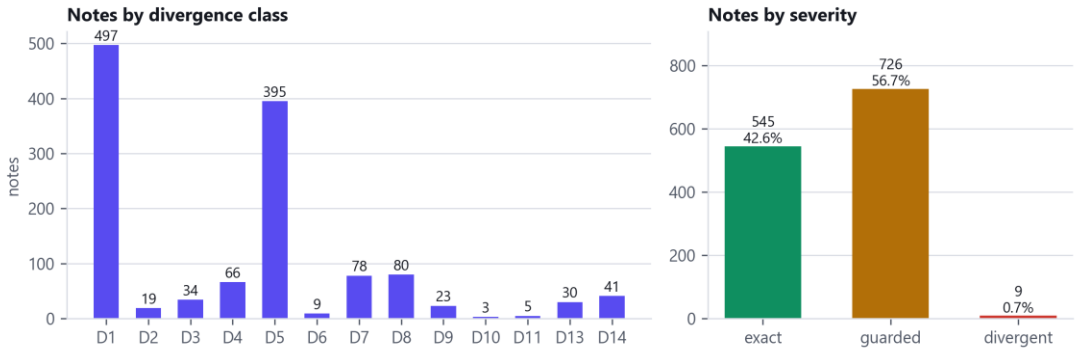


Figure 10.2. Fidelity notes by divergence class (left) and severity (right).

10.4 Studio on whole projects

Each project ran through the real application path (the HTTP API the UI uses) on a clean server with no LLM key and a cold build cache. A function counts as *accelerated* only if every replayed call matched and its measured speedup was at least 1.1x. The row marked "before print fix" is the first TextLab run, whose output hash differed until commit `bd7e8ca`.

- **textlab**: 35 functions found, 9 accelerated, 2 declined, 22 not eligible; whole job 232 s; program 17.2 s to 6.5 s (2.64x), output hash identical.
- **project-euler (problem 14)**: 102 functions found, 1 accelerated, 21 declined, 64 not eligible; whole job 51 s; program 8.9 s to 0.8 s (11.58x), output hash identical.
- **project-euler (problem 10)**: 102 functions found, 1 accelerated, 21 declined, 64 not eligible; whole job 33 s; program 7.1 s to 1.3 s (5.43x), output hash identical.
- **project-euler (problem 30)**: 102 functions found, 1 accelerated, 20 declined, 64 not eligible; whole job 78 s; program 9.9 s to 8.8 s (1.13x), output hash identical.
- **textlab (before print fix)**: 35 functions found, 9 accelerated, 2 declined, 22 not eligible; whole job 238 s; program 17.4 s to 6.8 s (2.56x), output hash DIFFERENT.

Table 10.3. Studio runs, zero-LLM path, build cache cold.

Project	Funcs found	Verified / guarded	Declined	Not eligible	Geomean fn speedup	End-to-end
textlab	35	9 (2 / 7)	2	22	9.7x	2.64x
project-euler (problem 14)	102	1 (0 / 1)	21	64	21.3x	11.58x
project-euler (problem 10)	102	1 (0 / 1)	21	64	1.8x	5.43x
project-euler (problem 30)	102	1 (0 / 1)	20	64	1.3x	1.13x
textlab (before print fix)	35	9 (2 / 7)	2	22	9.6x	2.56x (output differs!)

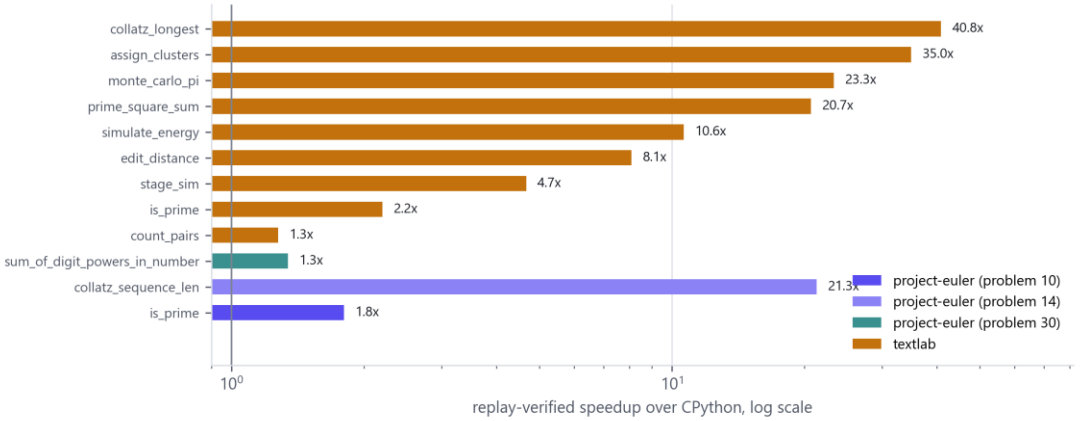


Figure 10.3. Per-function speedups from the Studio runs (replay-verified, log scale).

10.5 Reading the results

- **Where the speedup comes from:** unboxed int64 and double, contiguous monomorphic lists, no interpreter dispatch. The dictionary kernel shows what a slow runtime structure does to that.
- **Amdahl applies:** per-function gains exceed whole-program gains because the rest keeps running in Python.
- **The boundary can erase the gain:** the ranker declines such functions, and the replay's measured number is the final judge.
- **Threats to validity:** one laptop (Intel(R) Core(TM) Ultra 9 185H) and one toolchain family; whole-process timing; verification covers only replayed calls; most test programs were written by the team and its AI assistants, so they may share our blind spots.

11. Error handling, limitations and known issues

Compile-time problems print `line:col: Kind: message` with the source line and a caret; panic-mode recovery lets one run report several syntax errors. Compiled programs print Python's exception name and message (`IndexError`, `ZeroDivisionError`, `OverflowError`, `RecursionError`) so the oracle can compare them. In Studio a stage failure fails one function, never the job, and a compiled-code exception reaches Python as the matching exception or as `FallbackError`, which makes the shim run the original body.

11.1 Limitations with measured impact

Table 11.1. Limitations, their observed effect and the way forward.

Limitation	What happens	Observed impact	Remedy
C-extension libraries (NumPy, pandas, PyTorch, OpenCV)	Functions that use them are `not eligible`; they stay in Python.	Their own loops are already native; the speedup is lost only on the Python around them. Not measured on a NumPy project.	Call CPython from compiled code; a NumPy subset on Eigen or xtensor; LibTorch for PyTorch.
Dynamic typing and metaprogramming	`eval`, `exec`, `getattr`, a variable that changes type, int-or-float results are rejected with a reason.	Snippet `coin_change` refused: 21 of 22 snippets pass. Converting it would print `3.0 -1.0` instead of `3 -1`.	Compile one version per observed type signature, or `std::variant` unions.
64-bit integers (D1)	Checked arithmetic raises `OverflowError`; an int beyond int64 at the boundary raises too.	`2 ** 62 * 2`: CPython prints 9223372036854775808, PYTHIA `[OverflowError]`. Library mode: list element 2**70 gives `OverflowError`.	Promote to a bignum when a check fires.
ASCII-only strings (D13)	Non-ASCII input to a compiled function raises `FallbackError`; the original Python runs.	`len('héllo')`: CPython 5, standalone `6`; library mode `FallbackError (shim then runs the Python body)`.	UTF-8 strings with code-point indexing.
Memory reclamation (D7)	Standalone binaries never free memory. Library mode frees everything allocated in a call when it ends.	RSS stayed at 28640 KB after 200000 calls that each built a 500-element list and string (before: 28640 KB).	Region allocator or reference counting for standalone code.
Dictionary speed	Specialised dicts use a linear scan per lookup.	String-keyed word count, about 2,000 keys: about 29 times slower than CPython (Table Table 10.2).	Hash index in `DICT_IMPL`.
Boundary cost	Arguments are copied in and mutated containers copied back.	A call to a trivial compiled function costs about 0.15 us against 0.038 us for `len`; a 100,000-element list costs about 1.18 ms per call. The ranker declines functions where this dominates (Table Table 10.3).	Zero-copy buffers, data kept on the C++ side, fusing neighbouring functions.
Verification is differential testing	Agreement on recorded and labelled generated calls only.	A path the traced run never took is untested; fuzzing found 0 mismatches in 150 programs, which bounds nothing about other programs.	Fuzz around recorded inputs, validate each pass, prove one pass formally.
Unsupported constructs	Generators, nested functions, decorators, `with`, `global`, `*args`, `async`, `match` (Section 11.3).	Reported per function with a reason; 0 crashes of the converter on the stress repositories recorded in `STATUS.md`.	Closures by lambda lifting; generators by state machines.
Randomness, files, OS, regex, threads	Refused with a reason (`explain.py`).	Programs that print random numbers cannot be compared with CPython unless seeded.	Reproduce CPython's Mersenne Twister for seeded use.
Real student-style programs	Menu-driven apps (library, bank, inventory) usually import sibling modules or mix value types in dicts and are refused.	Of 46 such single files from small GitHub repos, 2 converted whole and matched CPython (`hunt3_results.json`).	Cross-module conversion; unions for mixed-type dicts.
Compile-time latency	g++ runs once per module and is cached by content.	Library check: 15.05 s cold, 5.63 s for a new module with the runtime built, 27 ms on a cache hit. A cold Studio run took 51 s on the demo repository and 232 s on TextLab.	Ship prebuilt modules for known projects; build in the background.
Needs a C++ toolchain	Bundled on Windows (GCC 16.2); on Linux and macOS the user installs g++.	Windows installer 90.0 MiB.	Ship a compiler in the Linux and macOS packages too.
Python to C++ only	One front end, one back end.	WebAssembly back end dropped.	The IR is language-agnostic after the front end.

11.2 Known issues

- **Snippet `coin_change`** is refused (int and float on different paths): 21 of 22 snippets pass, not the 22 of 22 in `STATUS.md` (Section 9.2).
- **`docs/grammar.ebnf` is the v0.1 core** (33 named rules; Review 1 reported 41). The language has grown past it; later constructs are described by prose rules generated from the compiler source.
- **Stale text:** the D8 note still describes `setjmp`/`longjmp`, and the D7 note calls reclamation "the Review 3 target".
- **Dictionaries use a linear scan** (`runtime/pyrt.cpp:131`), about 29 times slower than CPython on our dictionary kernel.
- **Stress harness not re-run;** `STATUS.md` lists three older stress-harness bugs that we did not re-investigate.
- **A stale server is the usual cause of impossible-looking errors;** the launcher checks a build fingerprint, but a forgotten WSL server on the same port can still answer first.

12. Member-wise contribution

The first commit (84f1251, 4 Sept 2026) already holds the Review 2 code base as one commit, so git cannot attribute Review 1 and 2 modules to people; ownership follows the Review 1 responsibility matrix and the Owner: headers in the files. What git does show: 138 commits by the repository owner (135 under one address, 3 under a GitHub no-reply address), 3 by Copilot (the dictionary pull request), 1 by AaryanGupta01 and 1 by Jahnavi. Three pull requests from teammates' forks are the only non-owner work. The Studio work (17 to 19 Oct) was committed by the owner with AI coding assistants (Section 14); its split below is **proposed** and must be agreed by the team.

Table 12.1. Member-wise contribution.

Member	Ownership (Reviews 1 and 2)	Evidence	Integration and testing
Vishal Vivek 24BCE2377 lead	Front end (grammar, lexer with the off-side rule, parser, AST, diagnostics), command line, repository, integration. Review 3: the Studio engine, server, web UI and installer.	138 of 143 commits (git shortlog); every Studio commit (2f46833 to 95bbb1b, 17 to 19 Oct); merges of the three pull requests (086b6f8, 079bcb5, 4ef0102) and of eight workstream branches (for example c8ca929, 176f5d0). Front-end files are in the first commit 84f1251 with Owner: Member 1 headers.	Integrated all workstreams, kept the oracle green, wrote STATUS.md and DECISIONS.md; built and ran the installer.
Rujuta Mangesh Kulkarni 24BDS0459	Symbol table, bidirectional type checker and local inference (typecheck.py), negative corpus and diagnostic quality, dictionary support.	Pull request 2 from rujuta-student, merged as 079bcb5 (9 Sept): runtime/pydict.hpp (351 lines), tests/dict_test.cpp, five documents. The commits in that pull request are authored by Copilot (Co-authored-by Copilot App); the git record shows an AI assistant wrote them on her branch. typecheck.py and tests/negative are in the first commit.	Negative suite (88 programs, each tested for message text); type-checker soundness fixes found by the oracle.
Koliparthy Venkata Jahnavi 24BDS0132	Divergence catalogue and Semantic Fidelity Report (lower.py), pyrt runtime, interval-analysis guard elision (optimize.py), PySet.	Pull request 3 from Jahnavikoliparthy, merged as 4ef0102 (9 Sept): commit ddc975b by Jahnavi adds runtime/pyset.hpp (533 lines) and tests/set_test.cpp (366 lines). lower.py, optimize.py (header: Owner Member 3, Review 2 deliverable) and pyrt.c/.h are in the first commit.	Fidelity statistics and the guard-elision measurement in Section 10.
Aaryan Gupta 24BDS0134	C++ emitter, C11 to C++17 migration, differential oracle, benchmark harness, program generator, CI, corpus.	Pull request 1 from AaryanGupta01, merged as 086b6f8 (4 Sept): commit 9264cbb "Migrate backend from C11 to C++17" (6 files, +270/-48: emit_c.py, emit_cpp.py, pyrt.cpp, pyrt.hpp, oracle.py, cli.py). emit_c.py, tools/oracle.py, tools/fuzz.py, tools/fuzz_run.py (Owner: Member 4) and the first CI workflow are in the first commit.	Oracle (78/88), generator (150/150 in this session), CI (three jobs), kernel benchmark design.

Table 12.2. PROPOSED split of the Studio work by compiler phase; not evidenced by git.

Member	Proposed ownership of the Studio work (to be agreed)
Vishal Vivek	Studio shell: pipeline.py (the ten stages), server/app.py and bus.py (REST and WebSocket events), the React UI, launcher.py and the Windows installer.
Rujuta Mangesh Kulkarni	In-house type inference and eligibility: convert/infer.py, types.py, search.py, observe.py; scan.py and reasons.py (the compiler as judge, plain-English reasons).
Koliparthy Venkata Jahnavi	Fidelity and the boundary: the fidelity gutter and bottom panel, libmode.py and pyrt_bridge.hpp (marshalling, FallbackError, OverflowError, arena), rank.py (boundary-cost model).
Aaryan Gupta	Verification and measurement: replay.py, _replay_worker.py, codec.py, bench.py, stress.py, property_test.py, the oracle and CI.

13. Conclusion and future work

PYTHIA set out to answer one question: can a student-built compiler turn the slow parts of an ordinary Python project into native code *and* show why the result can be trusted? The work delivered a complete compiler pipeline (Reviews 1 and 2), and in Review 3 a system around it that scans, traces, ranks, types, converts, compiles, verifies, benchmarks and splices a real project, with a user interface that exposes every step and a Windows installer. The compiler agrees with CPython on all 78 valid test programs and rejects all 88 invalid ones, the in-house inference converts the untyped programs without an LLM, and the fidelity report names every place where the two languages could differ. Still open: the dictionary runtime is slow, the WebAssembly back end was dropped, and reclamation covers library mode only. On a third-party repository the system found the hot function, compiled it

(21.3x), declined its caller on a measured 0.81x, and sped the program up 11.6x with identical output; on other candidates it found nothing worth compiling and said why, which is the intended behaviour. The honest summary of the result is that verified conversion of a well-defined subset works, is useful on compute-heavy pure-Python code, and is bounded by the language subset, the 64-bit and ASCII choices, and the quality of the recorded calls.

13.1 What we learned

- A transpiler's front end is the easy half; the hard half is the contract with the host language: the boundary, overflow, changing types, and showing the output is right.
- Reporting beats hiding: making every divergence a positioned note turned an unanswerable question into a list, and caught a regression in this very session.
- A measurement harness is part of the product: the ranker, the 1.1x threshold and the cold cut-off exist because a function that compiles is not always worth compiling. Measuring also exposed our slow dictionaries and the unproven guard elision.

13.2 Future work

- **Hash-indexed dictionaries** (fixes the 29x slowdown), **bignum promotion** on overflow and **UTF-8 strings** to remove D1 and D13.
- **Boundary cost:** zero-copy buffers, data kept on the C++ side, fusing neighbouring hot functions.
- **Specialisation:** one compiled version per observed type signature, so int-or-float functions convert instead of being refused.
- **Verification depth:** fuzzing around recorded inputs, per-pass translation validation, a machine-checked proof of one pass.
- **Ecosystem:** call CPython from compiled code, a NumPy subset on Eigen, a VS Code extension, a CI mode that fails when a converted function stops matching, and the dropped WebAssembly back end.

Publication potential. The Semantic Fidelity Report, a positioned account of where a translation could differ, treated as a first-class output of a source-to-source compiler and paired with replay on recorded calls, is the publishable part. It would need a larger independent corpus, a comparison with Cython, mypyc and Codon on the same programs, and a study of replay's false-accept rate (for example by mutation testing).

14. References

1. Aho, A. V., Lam, M. S., Sethi, R. and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley.

2. Pierce, B. C. and Turner, D. N. (2000). Local type inference. *ACM Transactions on Programming Languages and Systems*, 22(1), 1-44.

3. Pratt, V. R. (1973). Top down operator precedence. *Proc. 1st ACM Symposium on Principles of Programming Languages*, 41-51.

4. Cousot, P. and Cousot, R. (1977). Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. *Proc. 4th ACM POPL*, 238-252.

5. McKeeman, W. M. (1998). Differential testing for software. *Digital Technical Journal*, 10(1), 100-107.

6. Yang, X., Chen, Y., Eide, E. and Regehr, J. (2011). Finding and understanding bugs in C compilers (Csmith). *Proc. PLDI*, 283-294.

7. Pnueli, A., Siegel, M. and Singerman, E. (1998). Translation validation. *Proc. TACAS*, LNCS 1384, 151-166.

8. Leroy, X. (2009). Formal verification of a realistic compiler. *Communications of the ACM*, 52(7), 107-115.

9. ISO/IEC (2017). *ISO/IEC 14882:2017, Programming languages: C++*.

Official documentation consulted for the comparison (accessed October 2026).

10. Cython documentation, "Cython - an overview". https://cython.readthedocs.io/en/latest/src/quickstart/overview.html

11. Numba documentation, "Overview" and "Glossary" (nopython mode, type inference). https://numba.readthedocs.io/en/stable/

12. PyPy, project home page (speed and compatibility statements). https://pypy.org/

13. mypyc documentation, "Introduction". https://mypyc.readthedocs.io/en/latest/introduction.html
14. Nuitka project, README and user manual. https://github.com/Nuitka/Nuitka , https://nuitka.net/doc/user-manual.html
15. Codon, README and documentation (Apache-2.0). https://github.com/exaloop/codon , https://docs.exaloop.io/
16. Pythran documentation. https://pythran.readthedocs.io/en/latest/
17. Shed Skin, README. https://github.com/shedskin/shedskin
18. LPython, project page (alpha status, backends). https://lpython.org/
19. Python documentation, `sys.monitoring` and `sys.setprofile`. https://docs.python.org/3/library/sys.monitoring.html

Reused open-source components.

20. pybind11 (BSD-3-Clause), seamless operability between C++11 and Python. https://github.com/pybind/pybind11
21. FastAPI (MIT), Uvicorn (BSD-3-Clause), Starlette (BSD-3-Clause), Pydantic (MIT), websockets (BSD-3-Clause).
22. React 18 (MIT), Vite 5 (MIT), TypeScript 5 (Apache-2.0), Tailwind CSS 3 (MIT), Zustand (MIT), Framer Motion (MIT), cmdk (MIT), xterm.js (MIT).
23. Monaco Editor and @monaco-editor/react (MIT). Inter and JetBrains Mono fonts (SIL OFL 1.1) via Fontsource.
24. Lucide icons, `lucide-react` 0.460.0 (ISC licence), used in the application and in the figures of this report.
25. MinGW-w64 GCC 16.2 (UCRT build) distributed by WinLibs (GPL with runtime-library exception). https://winlibs.com/
26. Inno Setup 6.7.3 (Jordan Russell and Martijn Laan; free licence). https://jrsoftware.org/isinfo.php
27. CPython 3.13 relocatable distribution via the NuGet `python` package (PSF licence).
28. Playwright for Python (Apache-2.0) and Matplotlib (PSF-based licence), used to produce this report's screenshots and charts; python-docx (MIT) to generate the .docx.

AI coding assistants. PYTHIA was developed with the help of AI coding assistants. The repository history records Anthropic Claude (Opus 5.5 and Sonnet 5.5, used through Claude Code) and GitHub Copilot as co-authors in `Co-authored-by` trailers on 105 of 143 commits, including the dictionary-library pull request. This report and the viva guide were also drafted with such an assistant from the repository and from commands run on the team's machine. The team is responsible for the content.

Appendix A. User manual

A condensed version of `USER_GUIDE.md`. Screens are in Appendix H.

1. **Install.** Run `PYTHIA-Studio-Setup-1.1.0.exe` (per-user, no administrator rights). Start **PYTHIA Studio** from the Start menu. Python, the C++ compiler and the web UI are inside the installer.
2. **First start.** A three-step setup checks the computer (Python, compiler, headers, data folder), offers an optional AI key (skip it: the in-house path needs none), and then offers *Replay sample run*, *Open the demo project* or the *Playground*.
3. **Bring code in.** *Open folder*, *Clone repository* (a public GitHub URL), drop a `.py` file, or use the Playground. Give an **entry command** such as `python main.py` so PYTHIA can trace the program, rank functions by measured time and record real calls.
4. **Watch the pipeline.** The stepper runs *Ingest, Scan, Trace, Rank, Fix, Compile, Verify, Benchmark, Splice, Export*. Click any stage for its raw output; the agent panel narrates each step.
5. **Review.** Each function has a status chip (*Verified, Guarded, Fixable, Declined, Failed, Not eligible*). Open one for the **Diff** (original against accelerated file), the **three-way view** (Python, PySub, C++ with the fidelity gutter) or the **Journey** (sixteen steps from original source to splice).
6. **Accept and export.** Accept or reject per function (or accept all verified ones), then **Export** a folder or zip: your project with `_pythia_accel/`, `PYTHIA_REPORT.md` and `build_accel.sh`. Your original folder is never modified.
7. **Playground.** Type or paste a program; the C++ appears as you type. *Convert* turns plain Python into PySub with the in-house converter, and *Run* executes the original under CPython and the converted program compiled to C++ on the same input, reporting whether the output is identical.
8. **Command line.** `python -m pythia.cli prog.py --run` (compile and run), `--fidelity` (divergence report), `--emit-only` (print C++), `--mode leetcode` (a solution class without `main`), `--lib f,g` (build an importable module).

Appendix B. Installation guide

Table B.1. Installation paths.

You have	Do this
Windows 10 or 11	Run `PYTHIA-Studio-Setup-1.1.0.exe` from `dist/` (or the GitHub release); or unzip `PYTHIA-Studio-1.1.0-portable.zip` and run the launcher. Nothing else to install.
Linux, macOS, WSL	`./studio.sh` from a checkout, or the one-line installer `scripts/install.sh`, then `pythia-studio --app`. Needs Python 3.10 or newer, a C++17 compiler and `pip install pybind11`.
Docker	`docker build -t pythia-studio .` and `docker run --rm -p 127.0.0.1:8765:8765 -v pythia-data:/data pythia-studio`
Developers	`python -m pythia_studio --no-open --port 8797` (set `PYTHIA_HOME` to isolate state). UI development: `cd studio/web && npm install && npm run build` (the built `dist/` is committed).
Build the installer	`python packaging/windows/build.py` downloads the pinned CPython, WinLibs GCC and Inno Setup, assembles the tree and writes `dist/PYTHIA-Studio-Setup-1.1.0.exe`.

Troubleshooting. `pythia-studio --doctor` prints each check with its fix. If errors look impossible, check for a stale server: the doctor reports a build fingerprint and the launcher refuses a server with a different one; on this team's machines the usual culprit was an old server left running inside WSL and answering on the same port.

Appendix C. Sample runs

Per-function results of the Studio runs in Table Table 10.3, zero-LLM path. `Speedup` is the median over the replayed calls; `replay` is matched calls over replayed calls; `declined` rows show the measured probe speedup and the reason.

textlab

Table C.1. Functions compiled in `textlab`.

Function	File	Status	Speedup	Share	Replay	Note
`stage_sim`	main.py	guarded	4.7x	30.1%	20/?	
`monte_carlo_pi`	montecarlo.py	guarded	23.3x	12.5%	49/?	
`simulate_energy`	nbody.py	verified	10.6x	11.7%	49/?	
`edit_distance`	distance.py	guarded	8.1x	7.9%	500/?	
`prime_square_sum`	primes.py	guarded	20.7x	7.7%	50/?	
`assign_clusters`	kmeans.py	verified	35.0x	6.6%	80/?	
`collatz_longest`	collatz.py	guarded	40.8x	6.1%	49/?	
`life_generations`	life.py	declined	0.8x	5.9%	50/?	Compiled version measured 0.82x (slower than Python): marshalling cost outweighs
`is_prime`	primes.py	guarded	2.2x	5.3%	500/?	
`window_energy`	window.py	declined	0.3x	5.0%		Declined: marshalling ~3.668s (170520 calls x ~5003 elements) exceeds the ~1.163
`count_pairs`	bpe.py	guarded	1.3x	2.4%	80/?	

project-euler (problem 14)

Table C.2. Functions compiled in `project-euler (problem 14)`.

Function	File	Status	Speedup	Share	Replay	Note
`problem_014`	problem_014.py	declined	0.8x	98.3%	49/?	Compiled version measured 0.81x (slower than Python): marshalling cost outweighs
`collatz_sequence_len`	utils.py	guarded	21.3x	84.8%	500/?	
`problem_001`	problem_001.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_003`	problem_003.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_006`	problem_006.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_009`	problem_009.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_015`	problem_015.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_016`	problem_016.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`drop_spaces_or_hyphens`	problem_017.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_020`	problem_020.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`find_upper_bound`	problem_030.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_030`	problem_030.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`find_upper_bound`	problem_032.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`factorial`	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so

Function	File	Status	Speedup	Share	Replay	Note
gcd	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_leap_year	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_palindromic	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_pandigital	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_prime	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
lcm	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
sum_of_digit_powers_in_number	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
sum_of_digits_in_number	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so

project-euler (problem 10)

Table C.3. Functions compiled in `project-euler (problem 10)`.

Function	File	Status	Speedup	Share	Replay	Note
is_prime	utils.py	guarded	1.8x	72.3%	500/?	
problem_001	problem_001.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_003	problem_003.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_006	problem_006.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_009	problem_009.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_014	problem_014.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_015	problem_015.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_016	problem_016.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
drop_spaces_or_hyphens	problem_017.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_020	problem_020.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
find_upper_bound	problem_030.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_030	problem_030.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
find_upper_bound	problem_032.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
collatz_sequence_len	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
factorial	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
gcd	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_leap_year	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_palindromic	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_pandigital	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so

Function	File	Status	Speedup	Share	Replay	Note
lcm	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
sum_of_digit_powers_in_number	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
sum_of_digits_in_number	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so

project-euler (problem 30)

Table C.4. Functions compiled in `project-euler (problem 30)`.

Function	File	Status	Speedup	Share	Replay	Note
problem_030	problem_030.py	failed	n/a	99.3%	12/?	Replay mismatch (exception) on args -18: python raises ZeroDivisionError, compil
sum_of_digit_powers_in_number	utils.py	guarded	1.3x	52.4%	500/?	
find_upper_bound	problem_030.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (1 calls); the saving would be negligible, so
problem_001	problem_001.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_003	problem_003.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_006	problem_006.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_009	problem_009.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_014	problem_014.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_015	problem_015.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_016	problem_016.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
drop_spaces_or_hyphens	problem_017.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_020	problem_020.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
find_upper_bound	problem_032.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
collatz_sequence_len	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
factorial	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
gcd	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_leap_year	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_palindromic	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_pandigital	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_prime	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
lcm	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
sum_of_digits_in_number	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so

textlab (before print fix)

Table C.5. Functions compiled in `textlab (before print fix)`.

Function	File	Status	Speedup	Share	Replay	Note
`stage_sim`	main.py	guarded	4.1x	31.5%	20/?	
`monte_carlo_pi`	montecarlo.py	guarded	29.2x	14.2%	49/?	
`simulate_energy`	nbody.py	verified	10.7x	11.5%	49/?	
`edit_distance`	distance.py	guarded	9.8x	8.0%	500/?	
`prime_square_sum`	primes.py	guarded	21.9x	6.9%	50/?	
`assign_clusters`	kmeans.py	verified	26.8x	6.7%	80/?	
`collatz_longest`	collatz.py	guarded	31.3x	6.3%	49/?	
`life_generations`	life.py	declined	0.7x	5.9%	50/?	Compiled version measured 0.72x (slower than Python): marshalling cost outweighs
`window_energy`	window.py	declined	0.3x	5.0%		Declined: marshalling ~3.668s (170520 calls x ~5003 elements) exceeds the ~1.178
`is_prime`	primes.py	guarded	2.4x	4.8%	500/?	
`count_pairs`	bpe.py	guarded	1.3x	2.3%	80/?	

Appendix D. Complete test table

All 78 positive and 88 negative oracle programs from the run logged in Appendix F. Time columns are single-run, whole-process times and include process start-up.

Table D.1. Positive corpus (`tests/positive`).

Positive program	CPython	PYTHIA	Ratio	Result
t01_arith.py	96.2 ms	8.0 ms	12.0x	PASS
t02_lists.py	97.0 ms	9.5 ms	10.3x	PASS
t03_strings.py	100.7 ms	9.8 ms	10.3x	PASS
t04_control.py	113.3 ms	8.2 ms	13.7x	PASS
t05_floats.py	91.7 ms	9.0 ms	10.2x	PASS
t06_boolops.py	119.3 ms	9.0 ms	13.2x	PASS
t07_bubble.py	103.3 ms	6.8 ms	15.2x	PASS
t08_nested.py	87.5 ms	8.0 ms	10.9x	PASS
t09_sieve.py	109.9 ms	10.2 ms	10.8x	PASS
t100_linked_list_inferred.py	188.6 ms	7.3 ms	25.8x	PASS
t101_inheritance_dispatch.py	76.8 ms	7.6 ms	10.1x	PASS
t102_static_class_property.py	90.0 ms	8.5 ms	10.6x	PASS
t103_dunders_sorting.py	217.7 ms	11.6 ms	18.7x	PASS
t104_leetcode_solution.py	189.3 ms	9.0 ms	21.0x	PASS
t105_bst_stack_queue.py	214.3 ms	12.1 ms	17.7x	PASS
t106_dataclass_records.py	201.2 ms	14.1 ms	14.3x	PASS
t107_class_attrs_accounts.py	158.9 ms	10.9 ms	14.5x	PASS
t108_graph_tree_people.py	202.7 ms	12.5 ms	16.2x	PASS
t109_class_error_paths.py	242.5 ms	9.0 ms	27.0x	PASS
t10_collatz.py	830.2 ms	34.9 ms	23.8x	PASS
t110_private_inplace_membership.py	210.0 ms	12.5 ms	16.8x	PASS
t111_assert_and_nested_init.py	124.9 ms	8.7 ms	14.3x	PASS
t112_stdlib_heap_bisect_deque_copy_sys.py	495.2 ms	10.6 ms	46.9x	PASS
t113_stdlib_itertools_enum_setcomp.py	141.5 ms	10.6 ms	13.4x	PASS
t11_matmul.py	224.0 ms	14.4 ms	15.5x	PASS
t12_err_index.py	237.1 ms	12.3 ms	19.2x	PASS
t13_err_zerodiv.py	280.3 ms	11.6 ms	24.2x	PASS
t14_edges.py	140.7 ms	9.3 ms	15.2x	PASS
t15_classes_dict_nested_try.py	138.4 ms	7.8 ms	17.8x	PASS
t160_dict_get1.py	132.4 ms	7.6 ms	17.5x	PASS
t161_slice_assign_del.py	124.5 ms	7.8 ms	16.0x	PASS
t162_nlargest_str_container.py	125.8 ms	8.1 ms	15.4x	PASS
t163_try_else_finally.py	90.8 ms	7.3 ms	12.5x	PASS
t164_recursion_stdin.py	99.2 ms	16.3 ms	6.1x	PASS
t165_semicolons_inline.py	113.2 ms	8.2 ms	13.8x	PASS
t16_classes_methods_exceptions.py	121.6 ms	8.3 ms	14.7x	PASS
t17_loopvar_reassign_immune.py	104.4 ms	8.8 ms	11.9x	PASS
t18_pysub11_features.py	115.2 ms	12.6 ms	9.1x	PASS
t19_error_messages.py	129.3 ms	7.3 ms	17.6x	PASS
t20_ifexp_not.py	95.6 ms	9.3 ms	10.3x	PASS
t21_list_sorted_sources.py	120.7 ms	7.8 ms	15.5x	PASS
t22_ord_chr.py	210.6 ms	8.6 ms	24.5x	PASS
t23_chained_compare.py	124.2 ms	8.8 ms	14.2x	PASS
t24_tuples.py	118.2 ms	7.0 ms	17.0x	PASS
t40_list_dict_comprehensions.py	119.0 ms	9.1 ms	13.0x	PASS
t41_generator_consumers.py	112.2 ms	9.0 ms	12.5x	PASS

Positive program	CPython	PYTHIA	Ratio	Result
t42_enumerate_zip_reversed.py	120.3 ms	7.1 ms	17.1x	PASS
t43_key_functions.py	124.2 ms	11.2 ms	11.1x	PASS
t44_comprehension_edges.py	225.2 ms	12.6 ms	17.9x	PASS
t45_dict_changed_during_iteration.py	226.1 ms	8.2 ms	27.7x	PASS
t46_tuples_with_iteration.py	111.7 ms	7.9 ms	14.1x	PASS
t50_defaults_kwargs.py	150.8 ms	10.3 ms	14.7x	PASS
t51_optional_linked.py	224.5 ms	9.1 ms	24.8x	PASS
t52_isinstance.py	201.8 ms	6.9 ms	29.2x	PASS
t53_module_globals.py	223.6 ms	7.8 ms	28.7x	PASS
t54_sets.py	220.9 ms	7.4 ms	29.9x	PASS
t55_dict_values.py	215.3 ms	8.9 ms	24.2x	PASS
t56_optional_scalars.py	150.7 ms	10.8 ms	13.9x	PASS
t57_collections.py	244.3 ms	19.0 ms	12.8x	PASS
t70_bitops.py	150.0 ms	10.5 ms	14.2x	PASS
t71_math.py	135.4 ms	12.7 ms	10.7x	PASS
t72_slices.py	104.8 ms	8.1 ms	12.9x	PASS
t73_str_methods.py	122.2 ms	15.8 ms	7.7x	PASS
t74_list_dict_methods.py	103.8 ms	11.6 ms	8.9x	PASS
t75_pow_int_conv.py	110.6 ms	9.8 ms	11.3x	PASS
t76_format_percent.py	99.2 ms	8.1 ms	12.3x	PASS
t77_err_sqrt.py	228.0 ms	5.8 ms	39.0x	PASS
t78_err_dict_pop.py	188.9 ms	8.4 ms	22.5x	PASS
t79_err_negshift.py	182.0 ms	7.3 ms	25.0x	PASS
t80_err_pow_inverse.py	186.2 ms	7.8 ms	23.7x	PASS
t81_err_slice_step.py	251.1 ms	6.6 ms	38.3x	PASS
t82_err_log0.py	197.4 ms	8.7 ms	22.6x	PASS
t83_repr_concat.py	101.0 ms	7.7 ms	13.2x	PASS
t90_list_repeat.py	111.2 ms	7.9 ms	14.1x	PASS
t91_chained_assign.py	112.9 ms	7.5 ms	15.0x	PASS
t92_fstring_eq.py	116.8 ms	6.9 ms	17.0x	PASS
t93_cond_mixed_boolop.py	127.2 ms	8.0 ms	15.9x	PASS
t94_optional_tuple.py	147.7 ms	7.2 ms	20.6x	PASS

Table D.2. Negative corpus (`tests/negative`): the compiler must reject each with a diagnostic containing the expected text.

Negative program	Expected text	Result
n01_rebind.py	cannot assign	REJECTED
n02_badarg.py	expected int, found str	REJECTED
n03_undefined.py	is not defined	REJECTED
n04_return_type.py	returning str from a function declared -> int	REJECTED
n05_mixed_list.py	incompatible element types	REJECTED
n06_missing_return.py	without returning a value	REJECTED
n07_empty_list_infer.py	cannot infer element type	REJECTED
n08_arity.py	expects 2 argument	REJECTED
n09_bad_index.py	index must be int	REJECTED
n100_print_subclass_no_repr.py	defines neither __str__ nor __repr__	REJECTED
n101_multiple_inheritance.py	multiple inheritance	REJECTED
n102_override_signature.py	different signature	REJECTED
n103_classmethod_via_subclass.py	called through subclass	REJECTED
n104_field_not_inferable.py	cannot infer the type of field 'items'	REJECTED
n105_sort_without_lt.py	'<' not supported between instances of 'P' and 'P'	REJECTED
n106_operator_not_defined.py	unsupported operand type(s) for +	REJECTED

Negative program	Expected text	Result
n107_property_no_setter.py	has no setter	REJECTED
n108_frozen_dataclass.py	frozen dataclass	REJECTED
n109_base_defined_later.py	must be defined before	REJECTED
n10_str_item_assign.py	does not support item assignment	REJECTED
n110_eq_in_hierarchy.py	cannot define __eq__	REJECTED
n111_iter_dunder.py	__iter__ is outside the PySub subset	REJECTED
n112_exception_subclass.py	may only inherit from another class	REJECTED
n113_class_attr_call.py	must be a constant	REJECTED
n114_dataclass_mutable_default.py	mutable default	REJECTED
n115_method_only_in_subclass.py	has no method 'bark'	REJECTED
n116_unknown_decorator.py	decorator @functools.lru_cache	REJECTED
n117_super_outside_method.py	super() is supported only	REJECTED
n118_instance_hides_class_attr.py	hides class attribute	REJECTED
n119_default_repr_in_fstring.py	defines neither __str__ nor __repr__	REJECTED
n11_compare_types.py	cannot compare	REJECTED
n12_break_outside.py	outside loop	REJECTED
n13_class_no_init.py	no __init__	REJECTED
n14_missing_init_fields.py	no __init__	REJECTED
n15_bad_field.py	no field	REJECTED
n16_dict_bad_key.py	dict key type	REJECTED
n17_unknown_exception.py	unknown exception type	REJECTED
n18_print_class.py	no reproducible repr	REJECTED
n19_nested_list_too_deep.py	deeper than two levels	REJECTED
n20_break_in_try_no_loop.py	outside loop	REJECTED
n21_ifexp_mixed.py	single static result type	REJECTED
n22_chain_impure_middle.py	chained comparison with a computed middle operand	REJECTED
n23_tuple_var_index.py	tuple index must be an integer constant	REJECTED
n40_lambda_value.py	lambda is supported only as the key= argument	REJECTED
n41_len_generator.py	generator expressions are supported only as the sole argument	REJECTED
n42_enumerate_len.py	enumerate() is supported only as the iterable	REJECTED
n43_unpack_list.py	cannot unpack	REJECTED
n44_comp_var_scope.py	name 'x' is not defined	REJECTED
n45_set_comprehension.py	set comprehensions are outside the PySub subset	REJECTED
n46_range_value.py	range() is supported only as the iterable	REJECTED
n47_list_key.py	a key function must return int, float, bool, str or a tuple of those	REJECTED
n48_bare_genexp.py	Generator expression must be parenthesized	REJECTED
n49_comp_unpack_list.py	cannot unpack a list[int] into 2 names	REJECTED
n50_unknown_kwarg.py	unexpected keyword argument	REJECTED
n51_duplicate_kwarg.py	multiple values for argument	REJECTED
n52_missing_arg.py	missing	REJECTED
n53_nonconst_default.py	default values must be constants	REJECTED
n54_optional_arith.py	may be None here	REJECTED
n55_isinstance_float.py	cannot be decided statically	REJECTED
n56_print_set.py	hash order	REJECTED
n57_iterate_set.py	iterate over sorted	REJECTED
n58_import_os.py	is not supported in PySub	REJECTED
n59_typing_not_imported.py	is not imported	REJECTED
n60_unbound_local.py	UnboundLocalError	REJECTED
n61_none_infer.py	cannot infer a type	REJECTED
n62_union.py	unions other than	REJECTED
n63_set_pop.py	set.pop()	REJECTED

Negative program	Expected text	Result
n64_is_int.py	implementation-defined	REJECTED
n65_int_default_float.py	CPython keeps the	REJECTED
n66_global_undefined.py	must be assigned at module level	REJECTED
n67_defaultdict_factory.py	makes values of type int	REJECTED
n68_optional_len.py	may be None here	REJECTED
n70_bitop_float.py	bitwise operators need int or bool	REJECTED
n71_unsupported_module.py	is not supported in PySub	REJECTED
n72_partition.py	returns a tuple	REJECTED
n73_dict_keys_print.py	live view	REJECTED
n74_math_unknown.py	has no attribute	REJECTED
n75_format_nonliteral.py	string-literal template	REJECTED
n76_slice_dict.py	does not support slicing	REJECTED
n77_pct_d_str.py	real number is required	REJECTED
n79_invert_float.py	unary	REJECTED
n80_hash.py	not defined	REJECTED
n81_gcd_float.py	needs an integer	REJECTED
n82_pct_few.py	not enough arguments	REJECTED
n83_format_attr.py	outside the PySub subset	REJECTED
n84_format_impure.py	must be used exactly once	REJECTED
n85_fstring_ascii.py	!a	REJECTED
n93_boolop_value_mixed.py	unrelated types	REJECTED

Appendix E. Grammar specification and API specification

Grammar. `docs/grammar.ebnf` is reproduced as committed. It specifies the PySub v0.1 core; Section 11.4 explains how the implemented language extends it.

```
(* PySub v0.1 -- the statically typed Python subset accepted by PYTHIA *)
(* BCSE307L Team 2 / Project A15.  Owner: Member 1 (Vishal Vivek).          *)

module      = { NEWLINE | funcdef | statement } EOF ;

funcdef     = "def" NAME "(" [ params ] ")" [ "->" type ] ":" block ;
params      = param { "," param } ;
param       = NAME ":" type ;
type        = "int" | "float" | "bool" | "str" | "None"
            | "list" "[" type "]" ;

block       = NEWLINE INDENT statement { statement } DEDENT ;

statement   = if_stmt | while_stmt | for_stmt | simple_stmt
            | "return" [ expr ] NEWLINE
            | "pass" NEWLINE | "break" NEWLINE | "continue" NEWLINE ;

if_stmt     = "if" expr ":" block { "elif" expr ":" block } [ "else" ":" block ] ;
while_stmt  = "while" expr ":" block ;
for_stmt    = "for" NAME "in" ( range_call | expr ) ":" block ;
range_call  = "range" "(" expr [ "," expr [ "," expr ] ] ")" ;

simple_stmt = annassign | assign | augassign | exprstmt ;
annassign   = NAME ":" type "=" expr NEWLINE ;
assign      = target "=" expr NEWLINE ;
augassign   = target augop expr NEWLINE ;
exprstmt    = expr NEWLINE ;
target      = NAME | postfix "[" expr "]" ;
augop       = "+=" | "-=" | "*=" | "/=" | "//=" | "%=" | "**=" ;

(* Expressions are parsed by precedence climbing (Pratt).                   *)
(* Binding powers, loosest to tightest:                                     *)
(*   or(1) < and(2) < comparison(4) < +,-(6) < *,/,//,%(8) < **(12, right)  *)
expr        = or_expr ;
or_expr     = and_expr { "or" and_expr } ;
and_expr    = comparison { "and" comparison } ;
comparison  = sum [ ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) sum ] ;
sum         = term { ( "+" | "-" ) term } ;
term        = power { ( "*" | "/" | "//" | "%" ) power } ;
power       = unary [ "**" power ] ;                    (* right associative *)
unary       = ( "-" | "+" | "not" ) unary | postfix ;
postfix     = atom { "[" expr "]" | "." NAME "(" [ arglist ] ")" } ;
atom        = INT | FLOAT | STRING | "True" | "False" | "None"
            | NAME [ "(" [ arglist ] ")" ]
            | "(" expr ")"
            | "[" [ expr { "," expr } ] "]" ;
arglist     = expr { "," expr } ;

(* Lexical layer.  INDENT / DEDENT are synthesised by the scanner from      *)
(* leading whitespace (the off-side rule); blank and comment-only lines     *)
(* carry no indentation meaning.  Implicit line joining applies inside      *)
(* ( ) and [ ].                                                            *)
NAME        = ( letter | "_" ) { letter | digit | "_" } ;
INT         = digit { digit | "_" } ;
FLOAT       = INT "." { digit } [ exponent ] | INT exponent ;
STRING      = "'" { char } "'" | '"' { char } '"' ;
```

Server API (`pythia/studio/server/app.py`). All routes are served on 127.0.0.1 and reject foreign `Host` headers.

Table E.1. REST and WebSocket interface of the Studio server.

Method and path	Purpose
`GET /api/env`	compiler, pybind11, Python and LLM availability
`GET /api/demos`	bundled demo projects
`GET /api/settings`	read or change settings (`PUT` through the same path)
`POST /api/settings/test-key`	check an optional API key with a read-only call
`GET /api/doctor`	environment checks and build fingerprint
`POST /api/jobs`	list jobs; `POST` starts one (input: snippet, file, folder, git URL or demo)
`POST /api/upload`	upload a file or snippet

Method and path	Purpose
`GET /api/jobs`	list jobs; `POST` starts one (input: snippet, file, folder, git URL or demo)
`GET /api/jobs/{job}`	job state and summary
`GET /api/jobs/{job}/events`	events after a sequence number
`WS /api/jobs/{job}/ws`	WebSocket event stream (live or replay)
`POST /api/jobs/{job}/cancel`	cancel a running job
`GET /api/jobs/{job}/functions`	function summaries
`GET /api/jobs/{job}/function`	one function record, optionally with stage dumps
`POST /api/jobs/{job}/decision`	accept or reject a function
`GET /api/jobs/{job}/file-diff`	original against accelerated file
`POST /api/jobs/{job}/export`	write the accelerated project
`GET /api/jobs/{job}/export.zip`	download it as a zip
`GET /api/jobs/{job}/report`	printable summary
`GET /api/workspaces/recent`	recent workspaces
`GET /api/workspaces/{ws}/tree`	file tree of a workspace
`GET /api/workspaces/{ws}/file`	file contents
`GET /api/fs/list`	browse local folders
`POST /api/playground/compile`	PySub to C++ with diagnostics and fidelity notes
`POST /api/playground/run`	run original under CPython and the compiled version
`POST /api/playground/convert`	in-house conversion of plain Python to PySub
`GET /api/playground/convert/status`	progress of a conversion
`POST /api/stages`	tokens, AST, typed AST, PIR, C++ for a snippet

Appendix F. Evidence register

Every number in the report is traceable to a row here. Commands were run from the repository root at commit `95bbb1b` on the machine described in Table Table 5.1; raw logs are in `docs/review3/src/evidence/`.

Table F.1. Evidence register.

#	Claim	Command	Result	Date
1	Oracle: compiled output equals CPython on every positive program	`wsl .venv/bin/python tools/oracle.py -v`	78/78	2026-10-24
2	Oracle: invalid programs rejected with the expected text	same	88/88	2026-10-24
3	Oracle single-run median speedup	same	15.2x (min 6.1x, max 46.9x)	2026-10-24
4	Fidelity notes over positive corpus	`python docs/review3/src/fidelity_stats.py`	1280 notes: exact 545, guarded 726, divergent 9	2026-10-24
5	Untyped basics, no LLM	`python scripts/check_basics.py --jobs 6`	91/91	2026-10-24
6	Snippets	`python scripts/check_snippets.py`	21/22; refused: coin_change	2026-10-24
7	`coin_change` regression bisected	`git worktree add` at `6423a29` and `210a599`, `check_snippets.py coin_change`	passes at `6423a29`, refused from `210a599`; old conversion prints `3.0 -1.0`, CPython `3 -1`	2026-10-24
8	Library mode matches CPython	`wsl .venv/bin/python scripts/check_libmode.py`	OK: 74 calls, 22 class calls; cold build 15.05 s, cached 27 ms	2026-10-24
9	Generated programs	`tools/fuzz.py --n 150 --seed 0` then `tools/fuzz_run.py`	150/150 identical, 0 mismatches	2026-10-24
10	Property tests	`python scripts/property_test.py`	50/50 identical	2026-10-24
11	Kernel benchmark and guard elision	`wsl .venv/bin/python docs/review3/src/bench_kernels.py`	median 10.8x, geometric mean 6.6x; outputs identical: True	2026-10-24
12	Edge cases, boundary cost, memory over many calls	`wsl .venv/bin/python docs/review3/src/edge_cases.py`	14 cases; RSS 28640 to 28640 KB over 200000 calls	2026-10-24
13	Studio runs, zero LLM	`python docs/review3/src/studio_runs.py ...` against a server started with empty `GROQ_API_KEY` and `ANTHROPIC_API_KEY`	5 runs; see Table Table 10.3	2026-10-24
14	Stress harness on public repositories	`python scripts/stress.py OWNER/REPO --limit N`	not run this session (cut)	2026-10-24
15	Lines of code, test counts, symbol line numbers	`python docs/review3/src/collect_facts.py`	38,834 lines in product code and tools (non-blank), 78/88 oracle programs	2026-10-24
16	Environment	`python docs/review3/src/collect_env.py`	Intel(R) Core(TM) Ultra 9 185H; Windows Python 3.13.16; WSL Python 3.14.4	2026-10-24
17	Commit counts and AI co-author trailers	`git shortlog -sn`; `git log --format=%B` for `Co-authored-by`	143 commits; 138 owner; 105 with an AI trailer	2026-10-24
18	Installer and zip sizes	`ls -l dist/`	90.0 MiB and 214.6 MiB (built 19 Oct; not rebuilt this session)	2026-10-24
19	Grammar rule count	regex over `docs/grammar.ebnf`	33 named rules	2026-10-24
20	No `settrace`, no Electron, no Hindley-Milner in the code	repository search (grep)	no matches in product code	2026-10-24
21	LLM is optional, not the default path	read `pipeline.fix_one`, `llm.make_client`	annotate, then deterministic inference, then LLM only if a key exists	2026-10-24
22	Competitor descriptions	official pages fetched: Shed Skin, Nuitka, PyPy, LPython, mypyc, Numba, Pythran	Figure 4.1 cells; Cython and Codon pages were not retrieved in full (Cython from general knowledge, Codon from a search snippet), so those two rows are lower confidence	2026-10-24
23	Emitted code listing	`python -m pythia.cli e.py --emit-only`	Listing 7.1	2026-10-24

Claims in the earlier draft that were corrected or removed.

Table F.2. Corrections to the teammate's draft.

Draft claim	Finding	Action
Register numbers 24BDS0102, 24BDS0118, 24BDS0127 and Vishal as 24BDS	Review 1 report: 24BCE2377, 24BDS0459, 24BDS0132, 24BDS0134.	Corrected on the cover and in Section 12.
"Hindley-Milner" type inference	`typecheck.py` is bidirectional local inference with one fixed type per variable; the converter uses a lattice fixpoint. No unification-based generalisation exists (searched the code).	Removed; Section 7.3 and 7.4 describe the real algorithms.
"Electron desktop shell"	The app is a FastAPI server plus a React page shown in an Edge/Chrome `--app` window by `launcher.py`; no Electron anywhere in the repository (searched).	Corrected in Sections 5 and 6.

Draft claim	Finding	Action
"zero memory leaks"	Standalone binaries never free memory (D7). Library mode frees per call; measured RSS flat over 200000 calls.	Replaced by the measured statement.
`sys.settrace`	The tracer uses `sys.monitoring` with a `sys.setprofile` fallback (`_tracer_boot.py:66`); `settrace` appears nowhere.	Corrected in Section 7.9.
`symbol_table.py`, `stages.py` ownership, `splice.py` in `pythia/`	`symbol_table.py` does not exist (scopes are in `typecheck.py`); `stages.py` exists but is a read-only stage dumper for the UI; `splice.py` is `pythia/studio/splice.py`.	File list rebuilt from the tree (Listing 8.1).
Runtime owned by Aaryan; `lower.py` and `optimize.py` plus `stages.py` for Jahnavi	The Review 1 matrix and file headers give the runtime and fidelity to Jahnavi; the emitter and oracle to Aaryan.	Section 12 follows the matrix and git.
Made-up commit messages ("feat(front): lexer indentation", ...)	`git log` has no such messages; the first commit is a single squash.	Replaced by the real commits in Appendix G.
Intel i7-13700H, Python 3.11, GCC 13.2	Measured environment is in Table Table 5.1 (Intel(R) Core(TM) Ultra 9 185H, Python 3.13.16 and 3.14.4, GCC 15.2.0 and 16.2).	Replaced.
Speedups 12.4x to 84.6x, geometric mean 28.7x; 8-row benchmark table	No such measurements exist in the repository or in our runs. Our kernels: 0.03x to 29.7x, geometric mean 6.59x.	Replaced by Table Table 10.2.
"Assembly confirms vectorization"; "100% bounds-check elision"	No assembly was inspected, and guards are elided only where provable (Section 10.2 gives counts).	Removed.
"6,000+ LOC Quanta"; "142 functions"	`demo/quanta` is about 3,800 lines (`DECISIONS.md`); function counts come from the Studio runs in Table Table 10.3.	Replaced.
"Zero-overhead" pybind11 splicing; "0% false positives"; "100% of divergences caught"	Marshalling copies data (measured); replay can miss unexercised paths.	Replaced by measured costs and by Section 11.
Figures 4.1, 6.1, 10.1 referenced but absent	Report now contains every figure it refers to; cross-references are generated.	Fixed.
Test-case table TC-01 to TC-10 and the emitted-code listing	The loop listing used names (`_t_ctr`) that the emitter does not produce, and TC-07 claimed assembly evidence.	Rebuilt from runs (Table Table 9.2, Listing 7.1).

Appendix G. Contribution log

From `git log` and `git shortlog` at the commit above. Merge commits are listed where they are the only evidence.

Table G.1. Commits by teammates and the main milestones.

Date	Commit	Author	What	Member (by Review 1 matrix)
2026-09-04	84f1251	Vishal Vivek	Inital commit	owner (first commit holds the Review 2 code base)
2026-09-04	9264cbb	AaryanGupta01	Migrate backend from C11 to C++17	Aaryan (M4)
2026-09-04	086b6f8	Vishal Vivek	Merge pull request #1 from AaryanGupta01/cpp17-backend	Aaryan (M4), merge of PR 1
2026-09-04	16450af	Copilot	Add PyDict: Python-compatible dictionary library for PYTHIA compiler	Rujuta (M2), authored by Copilot
2026-09-09	ddc975b	Jahnavi	Add Python-compatible set runtime library	Jahnavi (M3)
2026-09-09	079bcb5	Vishal Vivek	Merge pull request #2 from rujuta-student/rujuta-student-add-dict-library	Rujuta (M2), merge of PR 2
2026-09-09	4ef0102	Vishal Vivek	Merge pull request #3 from Jahnavikoliparthy/add-pyset-library	Jahnavi (M3), merge of PR 3
2026-09-10	533f875	Vishal Vivek	Final Cleanup Before Review 2	owner (Studio, proposed split in Section 12)
2026-10-17	2f46833	Vishal Vivek	Add PYTHIA Studio build prompt, contracts and decisions	owner (Studio, proposed split in Section 12)
2026-10-18	74bfa7d	Vishal Vivek	Run natively on Windows: toolchain discovery (bundled MinGW-w64), static .pyd builds, Windows process control	owner (Studio, proposed split in Section 12)
2026-10-18	cf2cfb8	Vishal Vivek	Playground: AI conversion of plain Python to PySub (type-checker loop), stdin for interactive programs, run or	owner (Studio, proposed split in Section 12)
2026-10-18	37acf6d	Vishal Vivek	Windows app: self-contained bundle (Python 3.13 + MinGW-w64 GCC 16), launcher exe, Inno Setup installer, first	owner (Studio, proposed split in Section 12)
2026-10-18	d344271	Vishal Vivek	WIP: in-house converter type lattice and inference engine	owner (Studio, proposed split in Section 12)
2026-10-19	0cad7e4	Vishal Vivek	fix(convert, emit, studio): resolve gaps for coin_change and group_anagrams, always enable in-house conversion	owner (Studio, proposed split in Section 12)
2026-10-19	8c0e1dc	Vishal Vivek	Basics gate: 90 untyped everyday programs + check_basics.py (in-house path, no LLM, compiled vs CPython)	owner (Studio, proposed split in Section 12)
2026-10-19	f14f559	Vishal Vivek	feat: add property-based differential test harness, leetcode mode, and modernize tooling	owner (Studio, proposed split in Section 12)
2026-10-19	e4a4804	Vishal Vivek	feat(studio): restore LLM conversion option and cleanup workspaces to working demos	owner (Studio, proposed split in Section 12)
2026-10-19	95bbb1b	Vishal Vivek	feat(packaging): bundle demos and generate portable zip and Windows installer	owner (Studio, proposed split in Section 12)

Appendix H. User-interface screenshots and additional diagrams

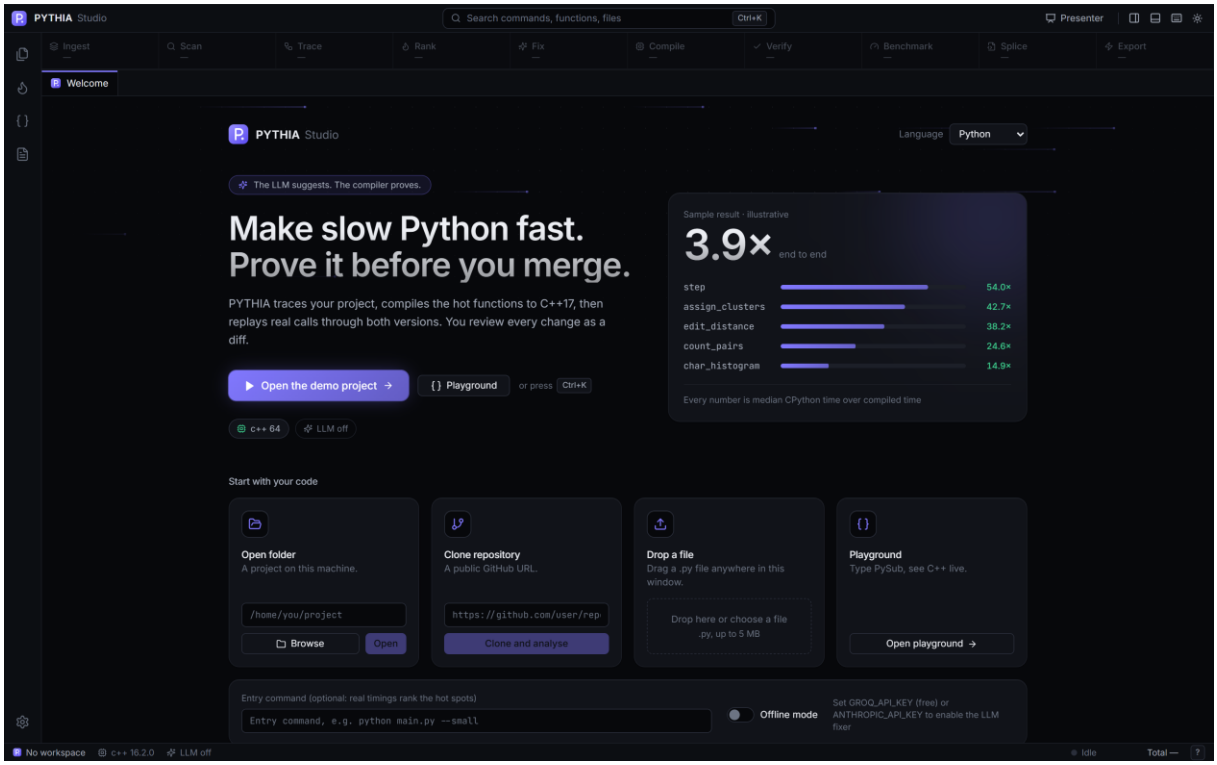


Figure H.1. Welcome screen: open a folder, clone a GitHub URL, drop a file, open the playground or replay a sample run.

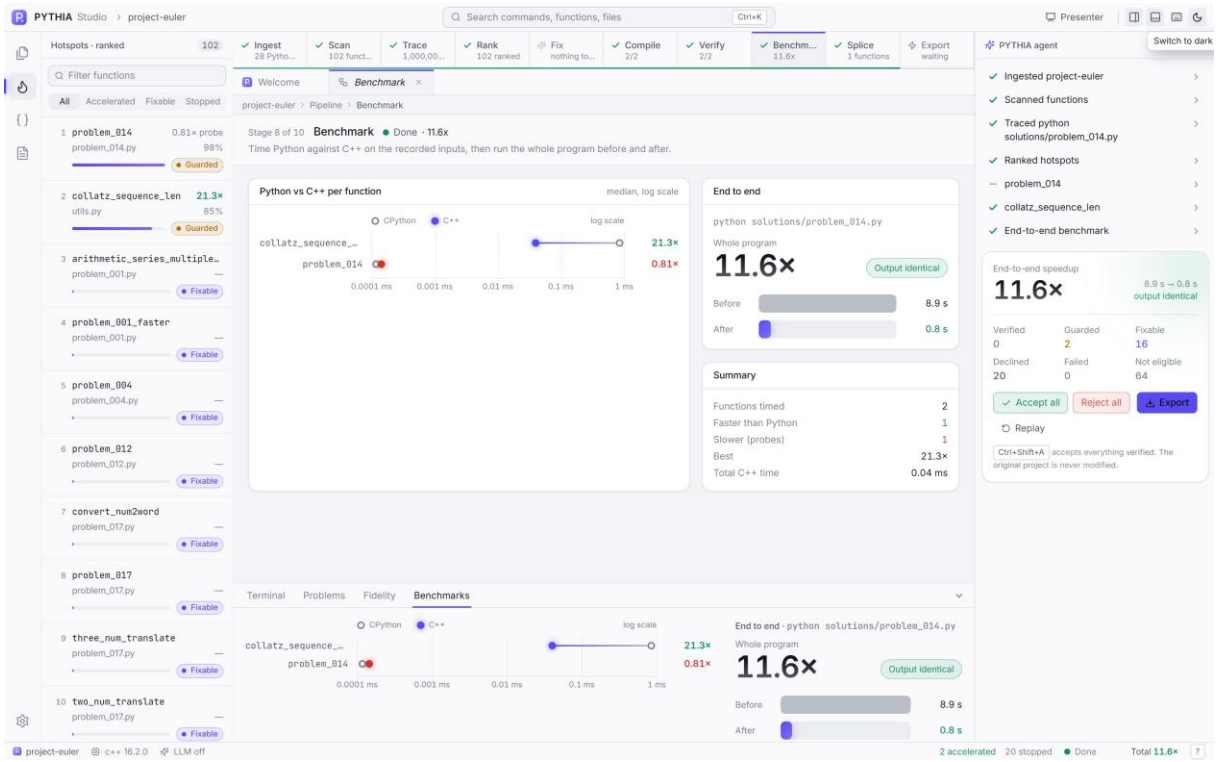


Figure H.2. A finished project run: the ten-stage pipeline stepper, ranked hotspots, agent narration and the end-to-end card.

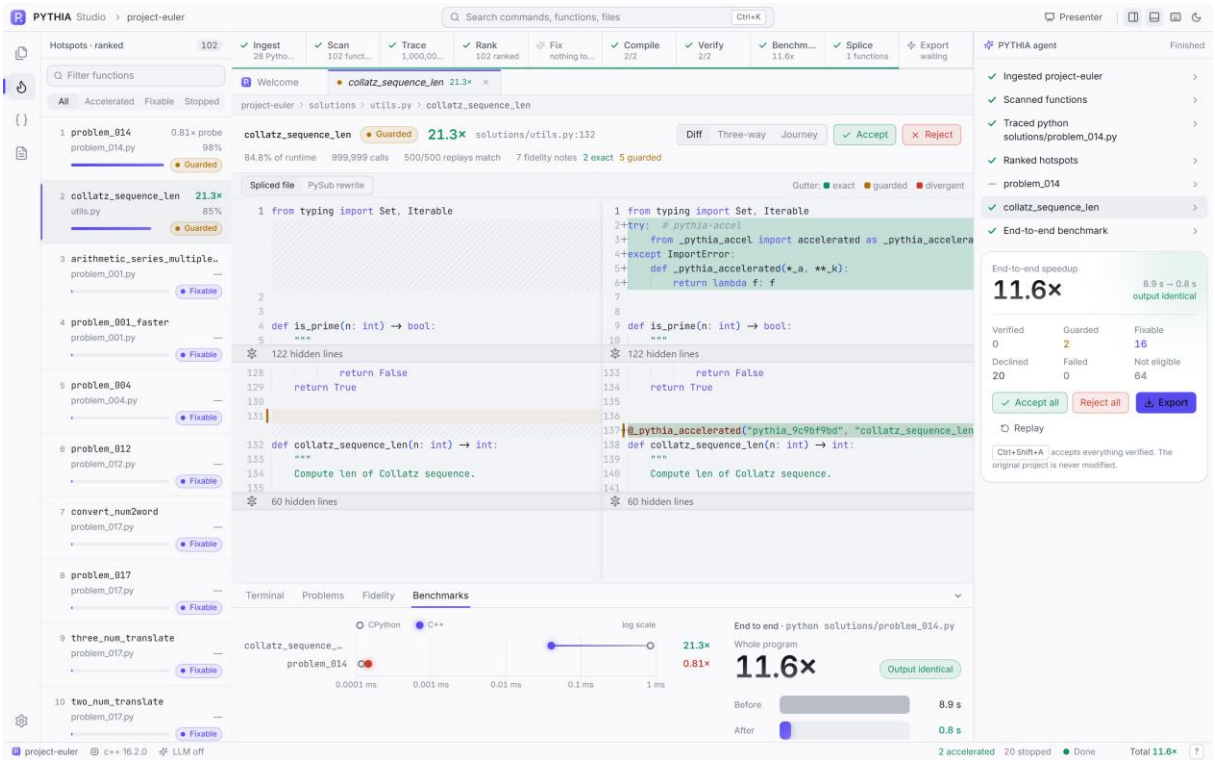


Figure H.3. Diff view with the fidelity gutter: one decorator line added above the original function, the body untouched.

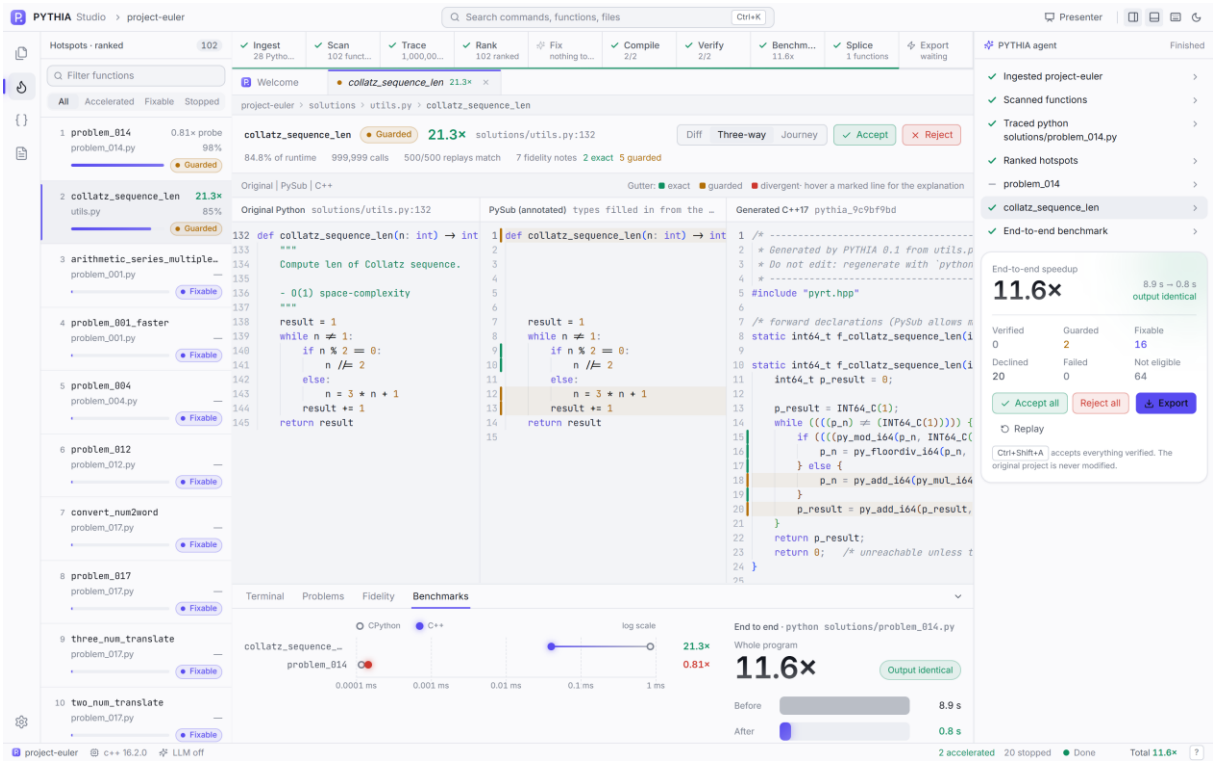


Figure H.4. Three-way view: Python, PySub and generated C++, with fidelity notes in the gutter.

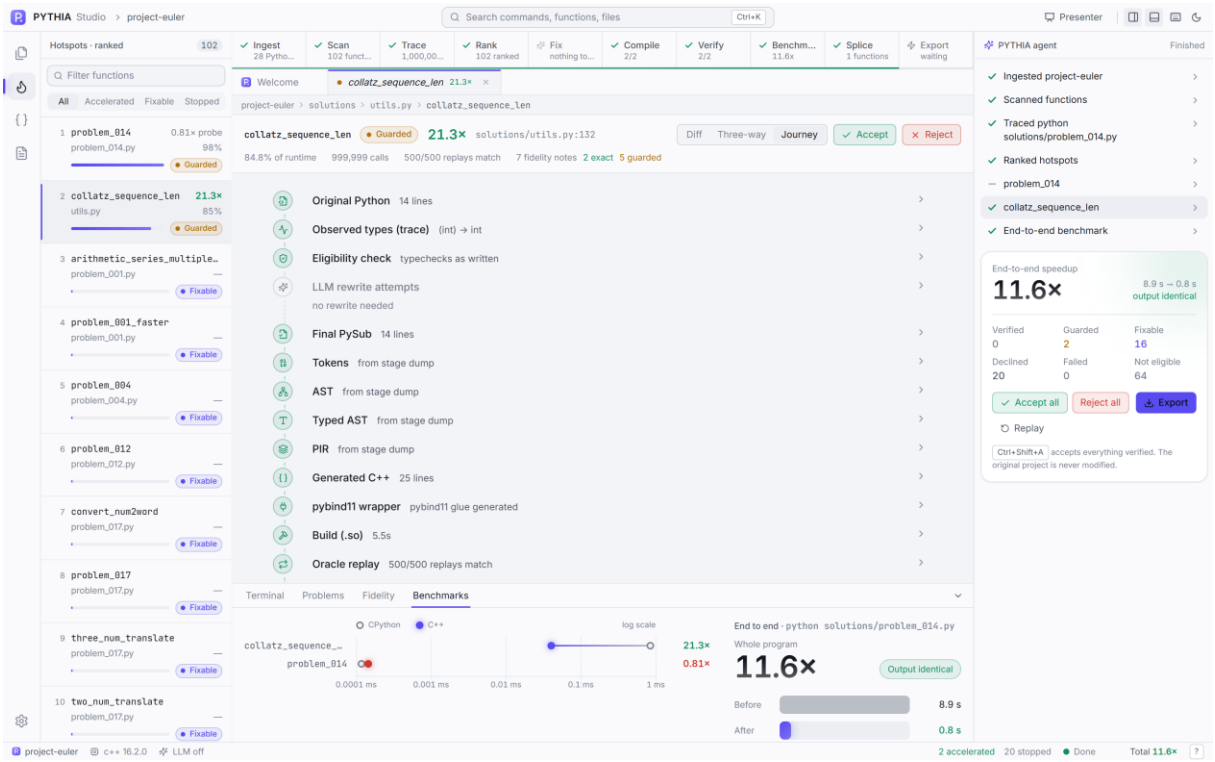


Figure H.5. The function journey: sixteen steps from original source to splice.

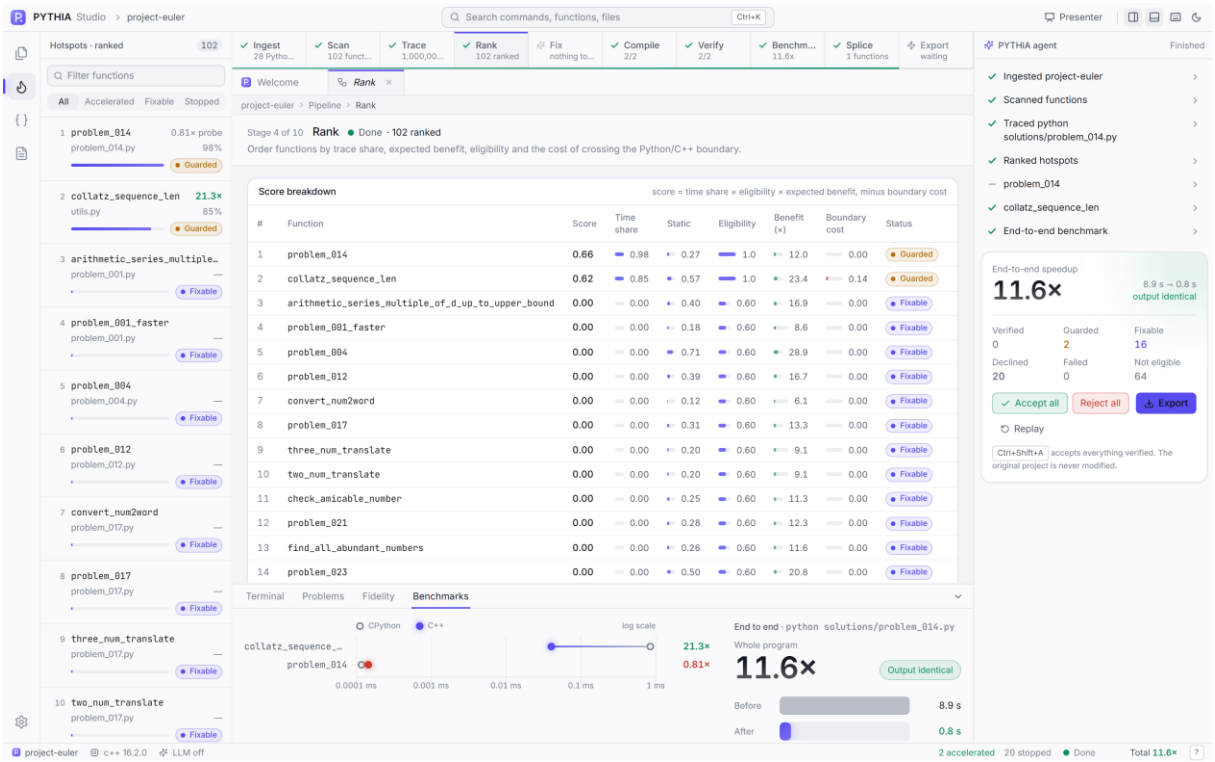


Figure H.6. Rank stage inspector: score breakdown per function.

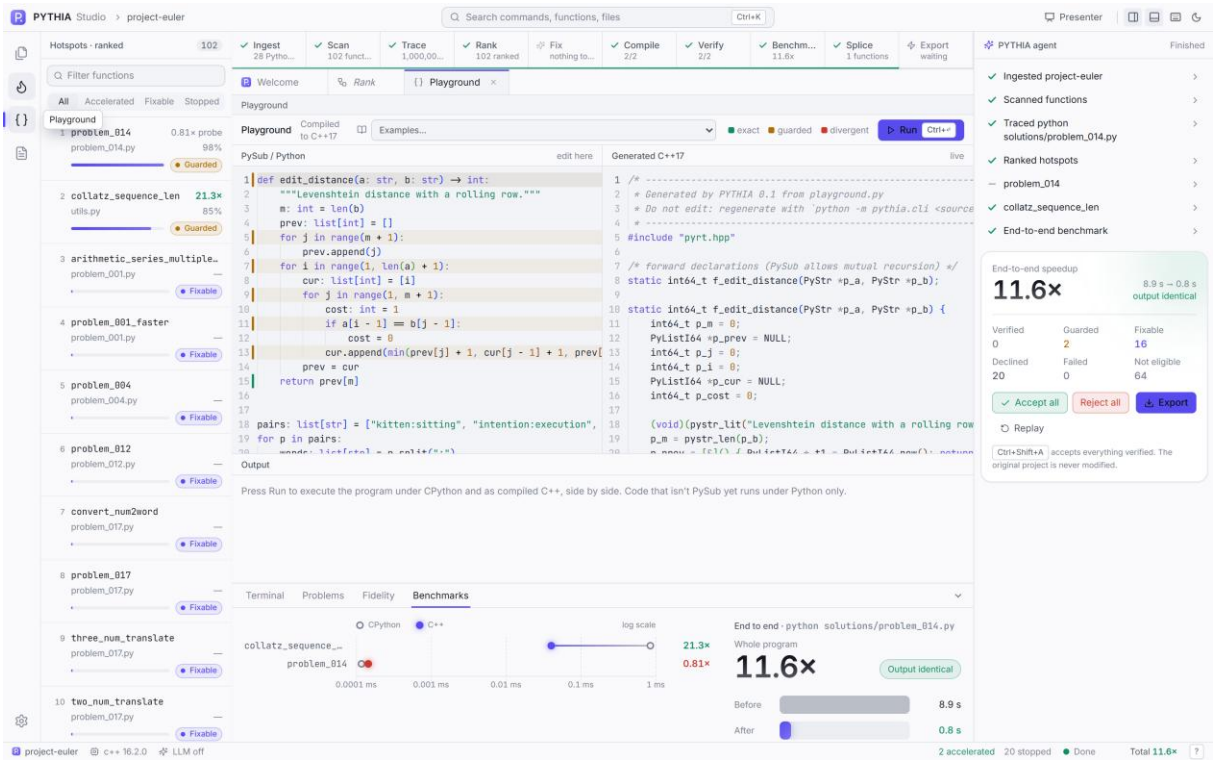


Figure H.7. Playground: convert, compile and compare against CPython.

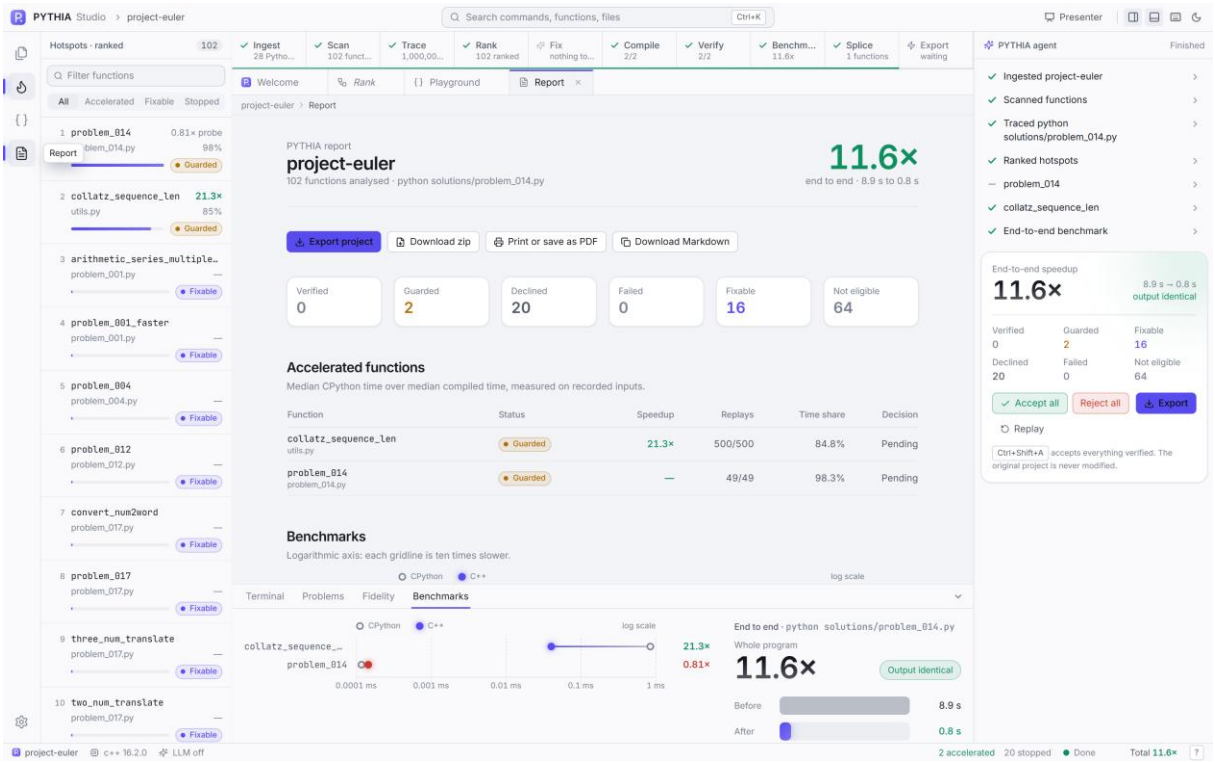


Figure H.8. Report page.